Appunti di Calcolo Numerico

Carbone Davide

A.A. 2024/2025

Indice

1 Aritmetica di Macchina **5**

1.1 Rappresentazione dei numeri (reali) 5
1.2 Arrotondamento di numeri reali a numeri di macchina 6
1.3 Operazioni di macchina ed equivalenze 9
1.4 Cancellazione numerica 10
1.5 Condizionamento di un problema numerico 12
1.6 Stabilità di un algoritmo 13

2 Fondamenti di Matlab **15**

2.1 Variabili vettori e matrici 15
2.2 Comandi di 'servizio' 21
2.3 Costrutti condizionali e iterativi 21
2.4 Operazioni aritmetiche e funzioni matematiche elementari 24
2.5 Funzioni 26
2.6 Grafica 27

3 Sistemi Lineari **33**

3.1 Norme 33
3.2 Forme speciali di matrici 35
3.3 Matrici a blocchi 37
3.4 Condizionamento di un sistema lineare 39
3.5 Metodi numerici per i sistemi lineari 42
3.5.1 Metodi numerici per sistemi diagonali 42
3.5.2 Metodi numerici per sistemi triangolari 43
3.5.3 Metodo della regola di Cramer, perché no in breve . . . 45
3.5.4 Metodo delle eliminazioni di Gauss 46

3.5.5 Fattorizzazione PA=LU 49
3.5.6 Applicazioni della fattorizzazione PA = LU 51
3.5.7 Fattorizzazione di Choleski 52
3.5.8 Fattorizzazione QR 53
3.5.9 Problema dei minimi quadrati (applicazione QR) . . . 54
3.5.10 Approssimazione di dati con polinomi di regressione (applicazione QR) 59

4 Valori singolari e scomposizione SVD 61
4.1 Decomposizione SVD 61
4.2 Applicazioni della SVD 64
4.2.1 Calcolo del rango di una matrice 64
4.2.2 Risoluzione di un sistema lineare 65
4.2.3 Risoluzione di un sistema lineare sovra-determinato . . 65
4.2.4 Risoluzione di un sistema mal condizionato 68
4.2.5 Risoluzione di un sistema lineare sotto-determinato . . 69

5 Autovalori 71
5.1 Cenni alla teoria degli autovalori e autovettori 71
5.2 Condizionamento del calcolo degli autovalori 74
5.3 Metodi numerici per il calcolo degli autovalori 75
5.3.1 Metodi per l'approssimazione di un solo autovalore . . 75

6 Analogie e differenze tra autovalori e valori singolari 83

Capitolo 1

Aritmetica di Macchina

Uno dei più grandi errori che si possono commettere quando si programma al calcolatore è quello di pensare che i risultati numerici ottenuti siano privi di errore.
Per motivare questa affermazione definiremo nel seguito cosa si intende per un'aritmetica di macchina e le problematiche ad essa collegate.

1.1 Rappresentazione dei numeri (reali)

Definizione 1.1.1 (Rappresentazione floating-point)**.** Si definisce rappresentazione floating-point di in numero reale a la seguente espressione:

$$a = (-1)^s p N^q, \quad s \in \{0, 1\}, \quad p \geq 0 \in \mathbb{R}, \quad q \in \mathbb{Z}$$

Definizione 1.1.2 (Rappresentazione floating-point **normalizzata**)**.** Una rappresentazione floating-point si dice **normalizzata** se p soddisfa:

$$N^{-}1 \leq p < 1$$

In questo caso p (mantissa) e q (esponente o caratteristica) sono univocamente determinati.

Una volta fissata una base N del sistema di numerazione, s, p e q quindi definiscono in modo univoco un numero reale a.
Un computer reale può memorizzare solo un numero finito di cifre per la mantissa e per l'esponente.

Non tutti i numeri reali sono allora rappresentabili con notazione floating-point.

Definizione 1.1.3 (Numeri di macchina)**.** Si definiscono **numeri di macchina** i numeri con mantissa ed esponente rappresentabili in un calcolatore. L'insieme dei numeri di macchina in un calcolatore che memorizza t cifre per la mantissa e q compreso tra L e U risulta così definito:

$$\mathbb{F} = \{0\} \cap \{(-1)^s 0.a_1 a_2 a_3 \dots a_t \cdot N^q, 0 \leq a_i < N, a_1 \neq 0, L \leq q \leq U\}$$

Definizione 1.1.4 (Aritmetica di macchina)**.** Un'**aritmetica di macchina** è l'insieme delle regole e operazioni matematiche tra numeri di macchina rappresentati in una certa notazione floating-point normalizzata.

Il più piccolo numero rappresentabile è $m = 0.1000\dots0 \cdot N^L$.
Si definisce **regione di underflow** l'insieme dei numeri reali diversi da 0 appartenenti a $(-m, m)$.
Il più grande numero rappresentabile è $M = 0.N-1N-1N-1\dots N-1 \cdot N^U$.
Si definisce **regione di overflow** l'insieme dei numeri reali appartenti a $(-\inf, -M) \cup (M, +\inf)$.
È importante notare che anche se i dati iniziali e il risultato finale sono numeri di macchina o arrotondabili a numeri di macchina le operazioni intermedie potrebbero dare origine a numeri che appartengono alla regione di overflow (valore resituito 0) o di underflow (valore restituito inf).

1.2 Arrotondamento di numeri reali a numeri di macchina

Se un numero reale possiede una mantissa normalizzata con più di t cifre il calcolatore ricorre ad una tecnica di arrotondamento chiamata "rounding to even".

Definizione 1.2.1 (Tecnica "rounding to even")**.** La mantissa p viene approssimata con la mantissa di macchina $\bar{p}$ più vicina e se p è equidistante da due mantisse di macchina consecutive allora p viene approssimata con quella delle due che ha l'ultima cifra pari.
In modo più formale se

$$p = a_1 a_2 \dots a_t a_{t+1} \dots a_n$$

e

$$p \in (\bar{p_1}, \bar{p_2})$$

con

$$\bar{p_1} = 0.a_1 a_2 \dots a_t \quad e \quad \bar{p_2} = \bar{p_1} + N^{-t}$$

due mantisse di macchina consecutive, allora la mantissa di macchina $\bar{p}$ corrispondente a $\bar{p}$s è data da

$$\bar{p} = \begin{cases} \bar{p_1}, \text{ se } p \in \left(\bar{p_1}, \frac{\bar{p_1}+\bar{p_2}}{2}\right) \text{ oppure } p = \frac{\bar{p_1}+\bar{p_2}}{2} \text{ e } a_t \text{ è pari} \\ \bar{p_2}, \text{ se } p \in \left(\frac{\bar{p_1}+\bar{p_2}}{2}, \bar{p_2}\right) \text{ oppure } p = \frac{\bar{p_1}+\bar{p_2}}{2} \text{ e } a_t \text{ è dispari} \end{cases}$$

Nel seguito $\bar{a}$ sarà il numero di macchina associato al numero reale a. È evidente che ogni volta che si memorizza $\bar{a}$ al posto di a si commette un errore.

Definizione 1.2.2 (Errore)**.** Sia $\tilde{x}$ un'approssimazione del numero x. Si definisce **errore assoluto** associato a $\tilde{x}$

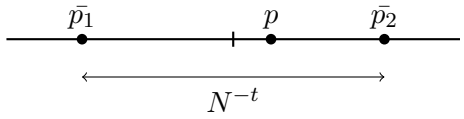
$$e_a = |x - \tilde{x}|$$

e **errore relativo**

$$e_r = \frac{|x - \tilde{x}|}{|x|}, \quad x \neq 0$$

Definizione 1.2.3 (Errore di arrotondamento)**.** Si definisce **errore di arrotondamento** l'errore che si commette quando si sostituisce un numero reale a con un numero di macchina $\bar{a}$.

Determiniamo ora una stima dell'errore assoluto $|a - \bar{a}|$ e dell'errore relativo $\frac{|a-\bar{a}|}{|a|}$ di arrotondamento.
Sia $a = (-1)^s p N^q$ con $N^{-1} \leq p < 1$ e $\bar{a} = (-1)^s \bar{p} N^q$.
Supponiamo che $p \in (\bar{p_1}, \bar{p_2})$, con $\bar{p_1}, \bar{p_2}$ mantisse di macchina consecutive.



Si ha allora necessariamente che

$$|p - \bar{p}| \leq \frac{1}{2} N^{-t}$$

L'errore assoluto risulta quindi

$$|a - \bar{a}| = |(-1)^s p N^q - (-1)^s \bar{p} N^q| = |p - \bar{p}| N^q \leq \frac{1}{2} N^{q-t}$$

e dal momento che $p \geq N^{-1}$

$$\frac{|a - \bar{a}|}{|a|} = \frac{|p - \bar{p}|}{|p|} \leq \frac{1}{2} N^{-t} \frac{1}{|p|} \leq \frac{1}{2} N^{1-t}$$

Definizione 1.2.4. Si definisce **epsilon di macchina** la quantità

$$eps = N^{1-t}$$

Si definisce **precisione di macchina** la quantità

$$\varepsilon_m = \frac{1}{2} N^{1-t}$$

La precisione di macchina è una costante specifica di ogni aritmetica floating-point, d'altronde dipende dalla base numerica del calcolatore (N) e dal numero di cifre con cui è rappresentata la mantissa (t).
Se chiamiamo l'errore relativo $\varepsilon = \frac{|a-\bar{a}|}{|a|}$ allora

$$\bar{a} = a(1 + \varepsilon), \quad |\varepsilon| \leq \varepsilon_m$$

MATLAB Matlab usa un'aritmetica di macchina con $N = 2$, la tecnica di arrotondamento "rounded to even" e 64 bit per la rappresentazione del numero di cui

1. 1 bit per il segno
2. 52 bit per la mantissa p
3. 11 bit per l'esponente q

Inoltre Matlab mette a disposizioni i seguenti valori

1. realmin fornisce il più piccolo numero di macchina (modulo)
2. realmax fornisce il più grande numero di macchina (modulo)
3. eps fornisce il valore dell'epsilon di macchina

1.3 Operazioni di macchina ed equivalenze

Tralasciando rare eccezioni il risultato di un'operazione tra due numeri di macchina **non** è un numero di macchina. Un calcolatore infatti non può eseguire esattamente le operazioni aritmetiche esatte ma può solamente effettuare le **operazioni di macchina**.

Definizione 1.3.1. Un'**operazione di macchina** $\odot$ associa a 2 numeri di macchina un terzo numero di macchina, ottenuto arrotondando il risultato della corrispettiva operazione in aritmetica esatta

Le operazioni $+, -, \times, /$ sono rappresentate dalle corrispettive operazioni di macchina $\oplus, \ominus, \otimes, \oslash$.
Per le operazioni di macchina rimane valida la proprietà commutativa:

$$\bar{a_1} \oplus \bar{a_2} = \bar{a_2} \oplus \bar{a_1}$$

$$\bar{a_1} \otimes \bar{a_2} = \bar{a_2} \otimes \bar{a_1}$$

ma non rimangono valide la proprietà associativa e distributiva:

$$\bar{a_1} \oplus (\bar{a_2} \oplus \bar{a_3}) \neq (\bar{a_1} \oplus \bar{a_2}) \oplus \bar{a_3}$$

$$\bar{a_1} \otimes (\bar{a_2} \otimes \bar{a_3}) \neq (\bar{a_1} \otimes \bar{a_2}) \otimes \bar{a_3}$$

$$\bar{a_1} \otimes (\bar{a_2} \oplus \bar{a_3} \neq (\bar{a_1} \otimes \bar{a_2}) \oplus (\bar{a_1} \otimes \bar{a_3})$$

$$(\bar{a_1} \otimes \bar{a_2}) \oslash \bar{a_3} \neq (\bar{a_1} \oslash \bar{a_3}) \otimes \bar{a_2}$$

Un altro fenomeno di cui bisogna essere a conoscenza è

$$\bar{a_1} \oplus \bar{a_2} = \bar{a_1}, \quad |\bar{a_2}| < |\bar{a_1}| eps$$

quando viene effettuata questa somma infatti le t cifre di mantissa vengono occupate prima con le cifre di $\bar{a_1}$ e poi con alcuni zeri, dal momento che le cifre significative di $\bar{a_2}$ dovrebbero essere collocate dopo la t-esima cifra della nuova mantissa.

$$\overbrace{0.1374623470000000000000000000000}^{\text{cifre mantissa di } \bar{a_1}}\overbrace{387423847}^{\text{cifre mantissa di } \bar{a_2}}$$

$$\underbrace{\hspace{6cm}}_{t \text{ cifre}}$$

Definizione 1.3.2 (Concetto di equivalenza)**.** Due quantità e_1 ed e_2 si definiscono equivalenti nell'aritmetica del calcolatore quando i loro valori calcolati dal calcolatore differiscono di una tolleranza relativa dello stesso ordine di grandezza o inferiore della precisione di macchina ε_m.

$$\frac{|\bar{e_1} - \bar{e_2}|}{|\bar{e_1}|} = O(\varepsilon_m)$$

1.4 Cancellazione numerica

Definizione 1.4.1 (Cancellazione numerica)**.** Siano $\bar{a_1} = (-1)^{s_1}\bar{p_1}N^{q_1}$, $\bar{a_2} = (-1)^{s_2}\bar{p_2}N^{q_2}$ le rappresentazione di macchina dei due numeri reali $a_1 = (-1)^{s_1}p_1N^{q_1}$, $a_2 = (-1)^{s_2}p_2N^{q_2}$.
Il fenomeno della **cancellazione numerica** consiste nella perdita di cifre della mantissa quando si effettua l'operazione di sottrazioni tra due numeri di macchina e sussistono le seguenti condizioni:

1. $s_1 = s_2$
2. $q_1 = q_2$
3. $\bar{p_1} \approx \bar{p_2}$
4. $\bar{p_1} \neq p_1$ e/o $\bar{p_2} \neq p_2$

Questo fenomeno si presenta perché in un'aritmetica finita, come abbiamo visto in precedenza, le operazioni effettuate sono operazioni di macchina e vengono effettuate su numeri di macchina. Se uno o più numeri di macchina coinvolti presentano una perturbazione l'operazione di macchina potrebbe ampliarla. Per capire meglio il fenomeno consideriamo il seguente esempio.

Esempio 1.4.1. Consideriamo i numeri $a_1 = 0.157824831$ e $a_2 = 0.157348212$.
In aritmetica esatta $a_1 - a_2 = 0.000476619 = 0.476619 \cdot 10^{-3}$.
Siano $\bar{a_1} = 0.15782$ e $\bar{a_2} = 0.15735$ le due rappresentazioni di a_1 e a_2 in un'aritmetica finita con $N = 10$ e $t = 5$.
In questa aritmetica $\bar{a_1} \ominus \bar{a_2} = 0.00047 = 0.47000 \cdot 10^{-3}$.
Come possiamo notare abbiamo perso 3 cifre dal risultato in aritmetica esatta.
Calcoliamo gli errori relativi su $\bar{a_1}$, $\bar{a_2}$ e $(\bar{a_1} \ominus \bar{a_2})$:

$$\frac{|a_1 - \bar{a_1}|}{|a_1|} = 0.3 \cdot 10^{-4} < \varepsilon_m$$

$$\frac{|a_2 - \bar{a_2}|}{|a_2|} = 0.1 \cdot 10^{-4} < \varepsilon_m$$

$$\frac{|(a_1 - a_2) - (\bar{a_1} \ominus \bar{a_2})|}{|a_1 - a_2|} = 0.1 \cdot 10^{-1} > \varepsilon_m$$

Nonostante gli errori relativi su $\bar{a_2}$ e $\bar{a_1}$ siano minori della precisione di macchina (e quindi $a_1 \sim \bar{a_1}$ e $a_2 \sim \bar{a_2}$) l'errore relativo sulla loro differenza calcolata in aritmetica di macchina è maggiore della precisione di macchina.

A volte è possibile manipolare l'espressione algebrica che identifica un problema numerico per evitare la cancellazione numerica.

Esempio 1.4.2. Consideriamo il problema numerico $y = \sqrt{x+\delta} - \sqrt{x}$. Se $|\delta| << x$ allora il problema di cancellazione numerica può essere eliminato razionalizzando:

$$y = (\sqrt[2]{x+\delta} - \sqrt{x})\frac{\sqrt{x+\delta} + \sqrt{x}}{\sqrt{x+\delta} + \sqrt{x}} = \frac{\delta}{\sqrt{x+\delta} + \sqrt{x}}$$

Se invece $\delta \approx -x$ il fenomeno della cancellazione numerica non può essere evitato.

Un altro modo per manipolare un'espressione algebrica del tipo $y = f(x)$ per evitare il fenomeno della cancellazione numerica è quello di sostituire ad $f(x)$ il suo sviluppo di Taylor centrato in un valore prossimo a quello di x.

Esempio 1.4.3. Consideriamo il problema numerico:

$$y = \frac{e^x - 1}{x}$$

Per $x \approx 0$ possiamo scrivere il problema come:

$$y = \frac{1}{x}(1 + x + \frac{x^2}{2!} + \ldots + \frac{x^n}{n!} + o(x^n) + 1 = 1 + \frac{x}{2!} + \ldots + \frac{x^{n-1}}{n!} + o(x^{n-1})$$

Ricordando il fenomeno per cui $\bar{a_1} \oplus \bar{a_2} = \bar{a_1}, \quad |\bar{a_2}| < |\bar{a_1}| eps$, possiamo limitarci a sommare i soli termini che danno un contributo reale in aritmetica finita.

Nello studio della propagazione degli errori bisogna distinguere il ruolo del problema dal ruolo dell'algoritmo utilizzato per risolverlo.
Nelle successive due sezioni daremo le definizioni di **condizionamento di un problema numerico** e di **stabilità di un algoritmo** e spiegheremo la loro relazione.

1.5 Condizionamento di un problema numerico

Definizione 1.5.1 (Condizionamento di un problema numerico)**.** Sia $y = f(x)$ un generico problema numerico e siano:

1. $\bar{x}$ una perturbazione dei dati x di input

2. $\bar{y}$ i risultati ottenuti dai dati $\bar{x}$ in aritmetica esatta

Un problema si dice **ben condizionato** se accade che l'errore relativo associato a $\bar{x}$ è dello stesso ordine o maggiore a quello associato a $\bar{y}$. In caso contrario si dice **mal condizionato**.

Possiamo studiare il condizionamento di un problema con stime del tipo:

$$\frac{|y - \bar{y}|}{|y|} \leq K(f, x) \frac{|x - \bar{x}|}{|x|}$$

Definizione 1.5.2. Si definisce **numero di condizionamento** del problema il più piccolo valore di $K(f, x)$ per il quale la relazione precedente risulta verificata.
Se K è abbastanza piccolo il problema risulta ben condizionato

In un problema ben condizionato quindi a perturbazioni nei dati corrispondono perturbazioni nei risultati dello stesso ordine di grandezza. In un problema mal condizionato invece a piccolo perturbazioni nei dati corrispondono grandi perturbazioni nei risultati.

1.6 Stabilità di un algoritmo

Definizione 1.6.1. Sia $y = f(x)$ un problema numerico generico. Data un'aritmetica finita siano inoltre:

1. $\bar{x}$ la rappresentazione di macchina dei dati x
2. $\bar{y}$ i risultati ottenuti attraverso un algoritmo a partire da $\bar{x}$ in aritmetica esatta
3. $\tilde{y}$ i risultati ottenuti attraverso un algoritmo a partire da $\bar{x}$ in aritmetica finita

L'algoritmo si dice **numericamente stabile** se accade che l'errore relativo

$$\frac{|\bar{y} - \tilde{y}|}{|\bar{y}|} = O(\varepsilon_m)$$

In caso contrario si dice **instabile**.

Capitolo 2

Fondamenti di Matlab

In questo capitolo illustreremo in modo non esaustivo i principali costrutti e le principali funzionalità del linguaggio MatLab. Lo scopo del capitolo quindi non è quello di fornire una conoscenza dettagliata del linguaggio ma quello di fornire gli strumenti per affrontare gli argomenti del corso. Si da inoltre per scontato che il lettore conosca già almeno un linguaggio di programmazione e sia dotato di quelle conoscenze di base dell'informatica trasversali ad ogni linguaggio.

2.1 Variabili vettori e matrici

MatLab è un linguaggio **matrix based** (il Mat nel nome sta infatti per Matrix e non per Math). Nonostante ciò si possono comunque dichiarare variabili scalari e vettoriali che però verranno trattate come matrici monodimensionali, 1x1 nel caso degli scalari e n x 1 nel caso dei vettori

```
MATLAB A = 1; % Definifisce una matrice 1x1 con il
numero 1 nell'unica entrata
V=[1;-1;1]; % Definisce un vettore (colonna) di 3 elementi
W=[3 4 5 6]; % Definisce una matrice 1 x 4 (riga)
M=[0 9 -2; 3 7 1; 5 1 0]; % Definisce una matrice 3 x 3
Z=[1 2 3 4 5]' % L'apice fa la trasposta di una matrice.  Z
sarà quindi un vettore (colonna)
```

Nota che in MatLab è necessario post-mettere ad ogni riga `;`. Ogni riga senza `;` stamperà il suo output in console. Nota inoltre che ogni carattere che segue il carattere `%` in MatLab sarà trattato come un commento.
Le matrici si possono costruire anche in maniera non convenzionale "annettendo" ad una matrice altre per formare una nuova matrice di dimensione compatibile

```
MATLAB N=[[M V; W; 0 0 0 0] Z]; % Forma una matrice 5
x 5 annettendo V a destra di M, W sotto e ancora sotto una
riga di 0.  Infine annette a destra Z
```

Le dimensioni di una matrice o di un vettori possono essere ricavate con la funzione `size(M)`

```
MATLAB sM=size(M); % Restituisce una matrice riga con le
dimensioni della matrice passata dalla più eterna (righe)
alla più interna (colonne):  [3 3]
```

Esistono delle funzioni che generano delle matrici o dei vettori con formati particolari (ricordiamo che MatLab non fa differenza tra matrici vettori e scalari).

```
MATLAB U=ones(3,2); % Matrice rettangolare 3 x 2 di 1
Z=zeros(3,4); %Matrice rettangolare 3 x 4 di 0
I=eye(4,4) %Matrice identità 4 x 4

% Se si passa un solo parametro a queste funzioni viene
generata in automatico una matrice quadrata U=ones(5);
Z=zeros(3);
I=eye(4);

v=ones(5,1); % Genera un vettore colonna
x = rand(4); % Genera una matrice 4x4 di numeri float
casuali da 0 a 1
```

Esistono diversi modi per creare **vettori (colonna)** oppure **matrici riga** con una dichiarazione **non esplicita** ma basata su intervalli

MATLAB
```
x=(0:0.1:0.5)'; % Genera un vettore di elementi
equidistanti con passo 0.1 partendo da 0 e arrivando al
massimo a 0.5 (se è possibile)

y=5:-2:-6; % Genera una matrice riga con valori che partono
da 5 e vengono incrementati (decrementati) di -2 (2) non
superando -6

% Se il passo è 1 si può omettere
d=-2:3; % Matrice riga di elementi equispaziati di passo 1

x=linspace(0,1,5); % Genera una matrice riga di elementi
equispaziati contenente 5 elementi in totale

y=logspace(-1,5,6); % Genera una matrice riga di elementi
equispaziati in scala log10 tra 10^-1 e 10^5 (6 elementi in
totale)
```

In MatLab si possono assemblare **matrici diagonali** attraverso funzioni built-in:

MATLAB

```
d0=1:5;
A=diag(d0,0); % Crea una matrice diagonale con diagonale d0
```

$$\begin{matrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 & 0 \\ 0 & 0 & 3 & 0 & 0 \\ 0 & 0 & 0 & 4 & 0 \\ 0 & 0 & 0 & 0 & 5 \end{matrix}$$

```
% Crea una matrice sommando 3 matrici generate con
diag(v , n).  n rappresenta l'ordine della diagonale
in cui mettere v.  0 -> principale, < 0 giù , > 0 su
B=diag(d0,0)+diag(-ones(4,1),1)+diag(3*ones(4,1),-1);
```

$$\begin{matrix} 1 & -1 & 0 & 0 & 0 \\ 3 & 2 & -1 & 0 & 0 \\ 0 & 3 & 3 & -1 & 0 \\ 0 & 0 & 3 & 4 & -1 \\ 0 & 0 & 0 & 3 & 5 \end{matrix}$$

```
% Aggiunge a B la diagonale 4 ( vertice alto-destra) e la
diagonale -3 (vettore di due elementi)
C=B+diag(7,4)+diag([-2 -5],-3);
```

$$\begin{matrix} 1 & -1 & 0 & 0 & 7 \\ 3 & 2 & -1 & 0 & 0 \\ 0 & 3 & 3 & -1 & 0 \\ -2 & 0 & 3 & 4 & -1 \\ 0 & -5 & 0 & 3 & 5 \end{matrix}$$

MATLAB

```
% Crea una matrice diagonale con tutte le entrate sulla
diagonale pari a π
D=pi*eye(7);
```

$$\begin{matrix} \pi & 0 & 0 & 0 & 0 \\ 0 & \pi & 0 & 0 & 0 \\ 0 & 0 & \pi & 0 & 0 \\ 0 & 0 & 0 & \pi & 0 \\ 0 & 0 & 0 & 0 & \pi \end{matrix}$$

Se il primo parametro della funzione `diag(A,n)` è una matrice estrae in un vettore la diagonale di ordine `n` di `A`

MATLAB

```
M = magic(5);

% Estrae la diagonale 1 di M
d1 = diag(M,1);
```

In MatLab l'indicizzazione di vettori e matrici parte da 1. È possibile effettuare selezioni sia per singola entrata (2 indici specifici) o per indici vettoriali / multi-indici.
Per entrambi i casi segnaliamo l'esistenza dell'operatore `end` che all'interno di una chiamata ad elemento di una matrice o di un vettore restituisce l'ultimo indice possibile.

MATLAB `M=magic(5); % matrice magic 5x5`

17	24	1	8	15
23	5	7	14	16
4	6	13	20	22
10	12	19	21	3
11	18	25	2	9

```
 c=M(2,3); % Restituisce il valore all'entrata 2 , 3 (7)

N=M([2 1],[3 4]); % Seleziona (nell'ordine specificato)
le entrate corrispondenti alle righe 2 ,1 e alle colonne 3 ,
4
```

7	14
1	8

```
 F1=M(5:-1:1,1:5); % Genera una matrice da M con le righe
invertite
F2=M(1:5,5:-1:1); % Genera una matrice da M con le colonne
invertite
M(2,3)=0; % Assegna all'entrata 2, 3 di M il valore 0
M(1:3,1:3) = ones(3); % Assegna alla sottomatrice
individuata dalla righe 1,2, 3 e dalle colonne 1,2,3 una
matrice quadrata di 1
```

1	1	1	8	15
1	1	1	14	16
1	1	1	20	22
10	12	19	21	3
11	18	25	2	9

Esiste inoltre la possibilità di utilizzare gli indici lineari. In Matlab, le

matrici vengono memorizzate seguendo l'ordine per colonna (column-major order). Questo significa che, quando si utilizza un indice lineare per accedere agli elementi di una matrice, Matlab la tratta come un vettore ottenuto concatenando le colonne una dopo l'altra. Formalmente, in una matrice di dimensioni m x n, si ha la seguente relazione tra gli indici lineari e gli indici di riga e colonna:

$$k = i + (j - 1) \cdot m$$

Segnaliamo inoltre l'esistenza di due funzioni per passare in automatico da un indice lineare agli indici di riga e colonna (`[i, j] = ind2sub(size(A), k);`) e viceversa (`k = sub2ind(size(A), i, j);`)

2.2 Comandi di 'servizio'

```
MATLAB  % Esiste l'fprintf esattamente speculare
all'omonima funzione in C
a = 3;
fout = fopen('example.txt' ,'w');
fprintf(fout , 'a:  %.2f' , a);
fclose(fout);
clc % pulisce lo schermo non pulisce la memoria
clear % pulisce la memoria ma non pulisce lo schermo
```

2.3 Costrutti condizionali e iterativi

In MatLab esitono due tipi di condizioni logiche: le condizioni logiche che operano elemento per elemento sulle entrate di una matrice e le condizioni logiche che operano solo su scalari (simili alle condizioni logiche in altri linguaggi).
Le prime (& e |), quando possibile, producono una matrice delle stesse dimensioni di quella valutata, in cui ciascun elemento è 0 se la condizione è falsa per il corrispondente elemento della matrice originale, altrimenti è 1.
Le seconde (&& e ||) producono un output binario basato sulla valutazione dell'espressione logica 'tradizionale'. Quest'ultime sono valutate ' a corto circuito', ovvero il secondo operando viene valutato solo se necessario (se in un and il primo operando è false il secondo non viene valutato).

Segnaliamo inoltre l'esistenza delle funzioni `all()` e `any()` che estendono l'utilizzo degli operatori logici per scalari alle matrici. In particolare `all()` ritorna vero se tutte le entrate di una matrice soddisfano la condizione e `any()` ritorna vero se almeno un'entrata della matrice soddisfa la condizione.

MATLAB
```
x=rand(4); % Genera una matrice 4x4 con float casuali da 0 a
1
x ≥ 0.5 & x< 0.8; % Genera una matrice 4x4 fatta di 0 ( se
la corrispondente entrata di x non soddisfa la condizione
logica ) e 1 ( se la corrispondente entrata di x soddisfa la
condizione)

z=[12 0 1];
c=[1 2];
z>0 | c≤1; % Produce un errore perché non ha senso produrre
alcuna matrice con la condizione logica passata
```

Di seguito riportiamo il generico schema di un costrutto `if-else if -else` in MatLab seguito da alcuni esempi di applicazione.

MATLAB
```
if <condizione>
......
elseif <condizione1>
......
else
......
end

x=rand(3,1);
if all(x>0.5)
end
```

MATLAB
```
n=5;
y = rand(7,1);
if n≤7 && y(n) ≤ 0.3
end

if n>7 || all(y<0.5)
end
```

Come fatto con il costrutto condizionale di seguito riportiamo il generico schema di un costrutto `while` e `for` in MatLab. Seguono alcuni esempi di applicazione

MATLAB
```
while <condizione>
......
end

% La variabile i prende ad ogni iterazione il valore
dell'elemento successivo nella matrice riga ( partendo dal
primo)
for i= <matrice riga>
......
end

% Alla variabile i viene assegnata ad ogni iterazione la
successiva colonna della matrice (partendo dalla prima a
sinistra)
for i= <matrice>
......
end
```

MATLAB

```
for k=1:5
k; % Assegna a k 1 , 2 ,3,4,5
end

w=[0 -3 12 -1];
for k=w
k; % Assegna a k 0 , -3 , 12, -1
end

A=[1 4 -1; 0 -1 -1];
for k=A
k; % Assegna a k 1 0 , 4 -1 , -1 -1
end

x=0:5; % Crea una matrice riga di elementi da 0 a 5 con
passo 1
x=x'; % Traspone la matrice riga creando un vettore colonna

% Il ciclo effettua una sola iterazione dal momento che il
vettore rappresenta un'unica colonna
for k=x
k; % Assegna a k il vettore colonna [0 1 2 3 4 5]'
end
```

2.4 Operazioni aritmetiche e funzioni matematiche elementari

Matlab include sia operatori per effettuare le operazioni sulle matrici sia operatori per effettuare operazioni elemento per elemento

MATLAB

```
A = rand(3,2);
B = rand(2,4);
C = A*B; % Calcola il prodotto matriciale tra A e B

x = [1 2 3];
y = [4; 5; 6];
s = x*y % Calcola il prodotto scalare tra x e y

x = [1 2 3];
y = [4 5 6];
v = cross(a,b); % Calcola il prodotto vettoriale tra x e y

A+B; % Da un errore perché le dimensioni delle matrici non
sono compatibili

D = rand(3,2);
E = A + D;
E = A-D;

A = rand(5);
A2 = A^2; % A*A

det(A); % determinante di A
inv(A); % inversa di A (non si usa mai, solo per matrici
piccolissime)
rank(A); % rango di A

% Funzioni elementari
x = 0.7;
y = sin(log(x)) * cos(x);

% Per effettuare la stessa operazione sulle singole entrate
di una matrice bisogna usare gli operatori .
x = [0.1 1.12 0.75];
y = sin(log(x)) .* cos(x);
y = x.^2; % quadrato di ciascun elemento di x
y = 3.^x;
y = 5./x;
w = [1 -4 0.3];
s = w./x; % Rapporto di ciascun elemento di w e il
corrispettivo di x.  solo se w e x hanno la stessa
dimensione
% Non esistono gli operatori .+ e .- in quanto + e - operano
già elemento per elemento
```

2.5 Funzioni

In Matlab si possono definire funzioni similmente a come si fa in altri linguaggi. In Matlab nel prototipo vanno definite sia le variabili in entrata sia le variabili in uscita(possono essere più di una e non è necessario ritornarle esplicitamente).

MATLAB
```
function [output1, output2, ...] = nome funzione(input1,
input2, ...)
...
end
```

Le funzioni dichiarate tramite `function` devono essere scritte in un apposito file `.m` contenente esclusivamente la dichiarazione della funzione. Hanno un loro spazio in memoria riversato (non condividono variabili con il resto del codice) e una volta salvato il file il compilatore le rende disponibili in tutti i file nella cartella del progetto.

Un altro metodo di dichiarare una funzione è quello di utilizzare le **function handle**

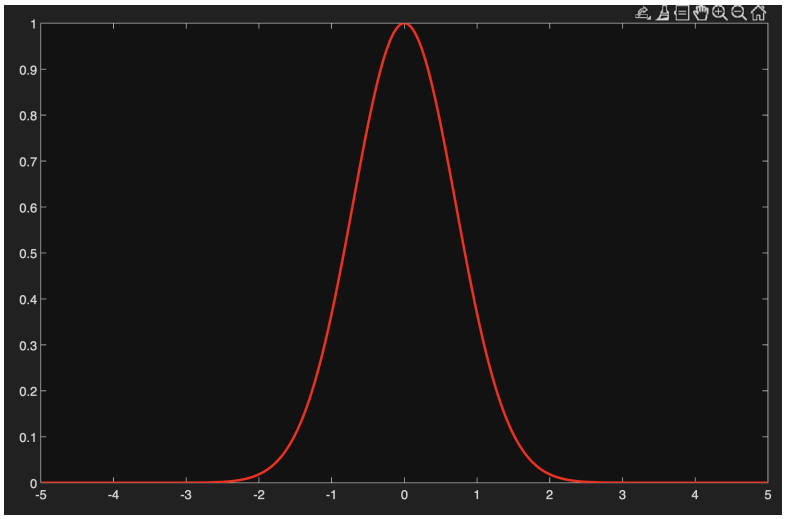
MATLAB
```
myfun = @(x) sin(x).*x.^2.*log(1+x.^4);
```

Tale dichiarazione è quasi equivalente a quella tramite `function`. La sostanziale differenza sta nel fatto che il corpo di una handle può essere formato solo da un'espressione e che la handle condivide lo stesso spazio in memoria del progetto, essendo salvata in una variabile nello script. Inoltre non è necessario definirla in un nuovo file.

2.6 Grafica

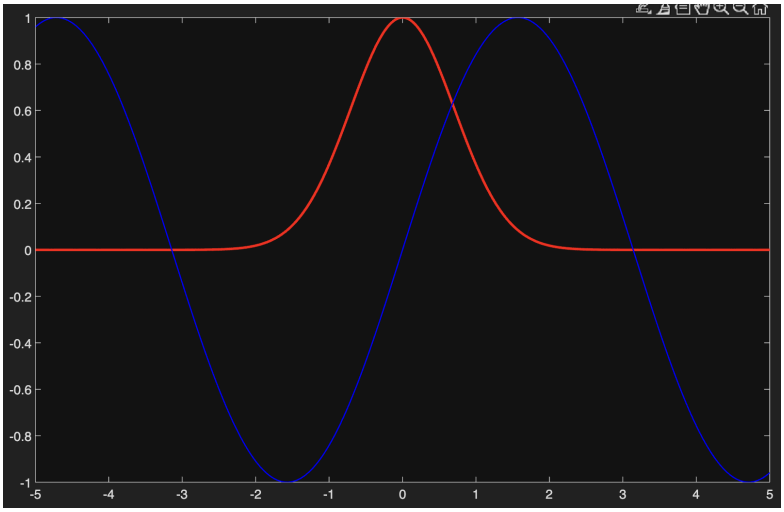
MatLab mette a disposizione una serie di comandi e funzioni utili per il plotting dei grafici. Il più basilare è il comando `plot()`:

MATLAB
```
x=linspace(-5,5,1000);
y = exp(-x.^2);
plot(x,y,'r-','linewidth',2)
```



In una stessa finestra si possono "plottare" più di un grafico con anteponendo a `plot` il comando `hold on`. Per disattivare la sovrapposizione basta inserire `hold off`.

MATLAB
```
x=linspace(-5,5,1000);
y=exp(-x.^2);
plot(x,y,'r-','linewidth',2)
hold on
plot(x,sin(x),'b-','linewidth',1)
hold off
```

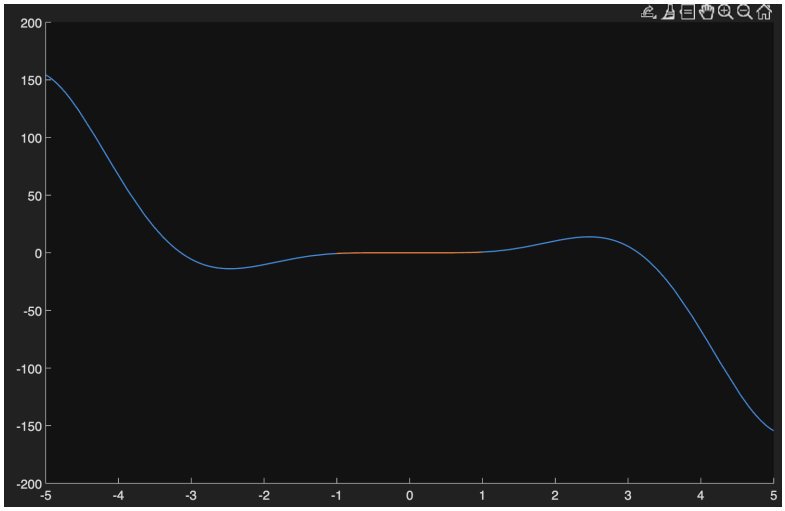


Si possono "plottare" anche function handle sia tramite plot sia tramite un'apposita funzione `fplot`.

MATLAB

```
myfun=@(x) sin(x).*x.^2.*log(1+x.^4);
t=linspace(-1,1,1000);
z=myfun(t);

figure % Apre una nuova finestra
hold on
fplot(myfun) % Fa il plot di una function handle decidendo
in automatico l'intervallo ( si può passare anche
l'intervallo desiderato )
plot(t,z)
hold off
```



Esistono comandi per "plottare" correttamente funzioni che cambiano il loro valore in diversi ordini di grandezza.

MATLAB

```
myfun1=@(x) sqrt((100*(1-1e-2*x.^2).^2+2e-2*x.^2)
./((1-x.^2).^2+0.1*x.^2));

x=linspace(0.1,100,10000);
y=myfun1(x);

figure
plot(x,y,'linewidth',2)

figure
loglog(x,y,'linewidth',2)

figure
semilogx(x,y,'linewidth' , 2)

figure
semilogy(x,y,'linewidth', 2)
```

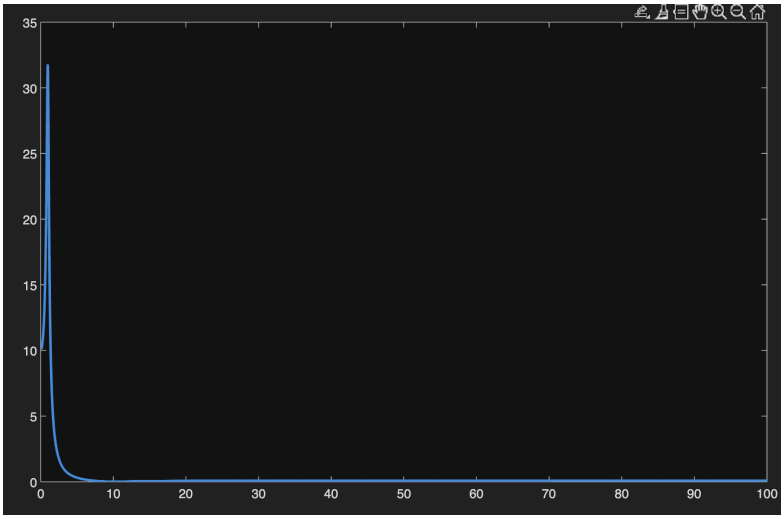


Figura 2.1: `plot`

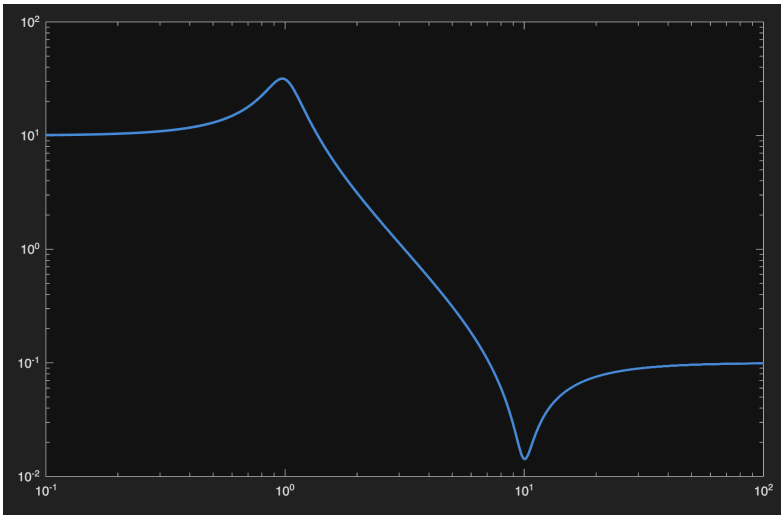


Figura 2.2: `loglog`

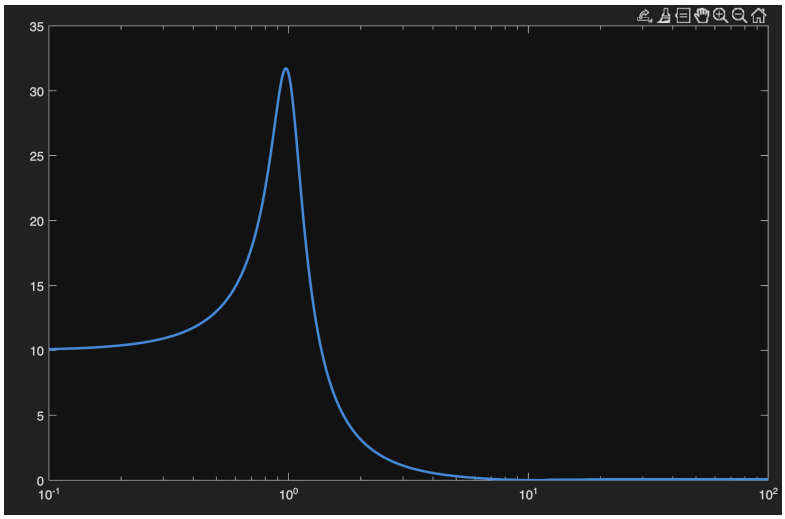


Figura 2.3: `semilogx`

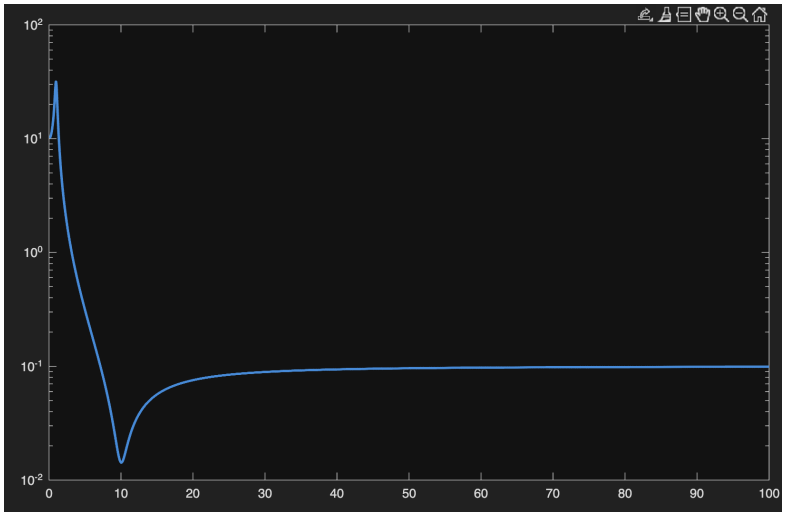


Figura 2.4: `semilogy`

Capitolo 3

Sistemi Lineari

Il fine di questo capitolo è quello di presentare al lettore i primi metodi numerici per la risoluzione di sistemi lineari. Prima di addentrarci nella trattazione dei metodi verranno presentate alcune nozioni riguardanti vettori e matrici.

3.1 Norme

Definizione 3.1.1 (Norma di un vettore)**.** Sia $\vec{v} \in \mathbb{R}^m$ un vettore.
Si definisce norma di un vettore una funzione $f : \mathbb{R}^m \to \mathbb{R}$ che rispetta le seguenti caratteristiche:

1. $f(\vec{v}) \geq 0 \quad \forall \vec{v} \in \mathbb{R}^m$ e $f(\vec{v}) = 0 \Leftrightarrow \vec{v} = 0$
2. $f(\alpha\vec{v}) = \alpha f(\vec{v}) \quad \forall \vec{v} \in \mathbb{R}^m, \quad \forall \alpha \in \mathbb{R}$
3. $f(\vec{w} + \vec{v}) \leq f(\vec{v}) + f(\vec{w})$

Nel seguito verranno considerate i seguenti tipi di norma di un vettore:

$$||\mathbf{x}||_1 = \sum_{i=1}^{n} |x_i|$$

$$||\mathbf{x}||_2 = \sqrt{\sum_{i=1}^{n} x_i^2} = \sqrt{x^T \cdot x} \quad \text{Norma euclidea}$$

$$||\mathbf{x}||_\infty = \max_{1 \leq i \leq n} |x_i|$$

MatLab fornisce funzioni per il calcolo delle norme:

MATLAB
```
norm(x,1); % Fornisce la norma 1 del vettore x
norm(x,2); % Fornisce la norma 2 del vettore x
norm(x); % Fornisce la norma 2 del vettore x
norm(x, inf); % Fornisce la norma infinita del vettore x
```

Il concetto di norma può essere esteso anche ad una matrice. Utilizzeremo le seguenti norme:
Sia $\mathbf{A} \in \mathbb{R}^{m,n}$

$$||\mathbf{A}||_1 = \max_{1 \leq j \leq n} \sum_{i=1}^{m} |a_{ij}|$$

$$\|\mathbf{A}\|_\infty = \max_{1 \leq i \leq m} \sum_{j=1}^{n} |a_{ij}|$$

Possiamo notare che la norma 1 prende il valore massimo tra le norme dei vettori colonna, mentre la norma infinita prende il valore massimo tra le norme dei vettori righe. Risulta quindi valida la seguente relazione:

$$||\mathbf{A}||_\infty = ||\mathbf{A}^T||_1$$

Segnaliamo l'esistenza della norma 2 di una matrice $||\mathbf{A}||_2$, detta norma spettrale, che verrà trattata in seguito.

MATLAB
```
norm(A,1); % Fornisce la norma 1 della matrice A
norm(A,inf); % Fornisce la norma infinita della matrice
A
```

Definizione 3.1.2 (**Compatibilità** della norma di matrice con la norma di vettore)**.** Una norma di matrice $||A||_p$ si dice compatibile con una norma di vettore $||x||_p$ quando sussiste la seguente relazione:

$$||Ax||_p \leq ||A||_p \cdot ||x||_p$$

In generale tale relazione è verificata per $p = \{1, 2, \infty\}$

3.2 Forme speciali di matrici

In questo paragrafo elencheremo una serie di forme "speciali" che possono avere le matrici.

Definizione 3.2.1 (**Triangolare superiore**)**.** Una matrice $\mathbf{A} \in \mathbb{R}^{m,n}$ si definisce **triangolare superiore** se $a_{ij} = 0, \quad i > j$

$$\mathbf{A} = \begin{pmatrix} * & * & * \\ 0 & * & * \\ 0 & 0 & * \end{pmatrix}$$

Definizione 3.2.2 (**Triangolare inferiore**)**.** Una matrice $\mathbf{A} \in \mathbb{R}^{m,n}$ si definisce **triangolare inferiore** se $a_{ij} = 0, \quad i < j$

$$\mathbf{A} = \begin{pmatrix} * & 0 & 0 \\ * & * & 0 \\ * & * & * \end{pmatrix}$$

Definizione 3.2.3 (**Diagonale**)**.** Una matrice $\mathbf{A} \in \mathbb{R}^{m,n}$ si definisce **diagonale** se $a_{ij} = 0, \quad i \neq j$

$$\mathbf{A} = \begin{pmatrix} * & 0 & 0 \\ 0 & * & 0 \\ 0 & 0 & * \end{pmatrix}$$

Definizione 3.2.4 (**Tridiagonale**)**.** Una matrice $\mathbf{A} \in \mathbb{R}^{m,n}$ si definisce **tridiagonale** se $a_{ij} = 0, \quad |i - j| > 1$

$$\mathbf{A} = \begin{pmatrix} * & * & 0 \\ * & * & * \\ 0 & * & * \end{pmatrix}$$

Definizione 3.2.5 (Trasposta)**.** Si definisce **trasposta** della matrice $\mathbf{A}$ la matrice $\mathbf{A^T}$ le cui entrate sono definite come $a^T_{ij} = a_{ji}$

Definizione 3.2.6 (Simmetrica)**.** Una matrice $\mathbb{A}$ si dice **simmetrica** se $\mathbf{A} = \mathbf{A^T}$

Definizione 3.2.7 (Inversa)**.** Si definisce **inversa** della matrice $\mathbf{A} \in \mathbb{R}^{n,n}$ la matrice $\mathbf{A^{-1}}$ tale che $\mathbf{AA^{-1}} = \mathbf{A}^{-1}\mathbf{A} = \mathbf{I}$. Non sempre si può definire l'inversa.

Definizione 3.2.8 (Ortogonale)**.** Una matrice $\mathbf{A}$ si dice **ortogonale** se $\mathbf{A}\mathbf{A}^{\mathbf{T}} = \mathbf{A}^T\mathbf{A} = \mathbf{I}$. Pertanto l'inversa di una matrice ortogonale è la sua trasposta.

Definizione 3.2.9 (Matrice di permutazione)**.** Si definisce **matrice di permutazione** una matrice ottenuta permutando le righe della matrice identità. Moltiplicando a sinistra una matrice o un vettore per una matrice di permutazione si applica la permutazione alla matrice o al vettore.

Esempio 3.2.1. Consideriamo la seguente matrice di permutazione

$$\mathbf{P} = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{pmatrix}$$

calcoliamo i prodotti $\mathbf{PA}$ e $\mathbf{Pb}$, con $\mathbf{A} \in \mathbb{R}^{3,3}$ e $\mathbf{b} \in \mathbb{R}^3$

$$\mathbf{PA} = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{pmatrix} \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix} = \begin{pmatrix} a_{31} & a_{32} & a_{33} \\ a_{21} & a_{22} & a_{23} \\ a_{11} & a_{12} & a_{13} \end{pmatrix}$$

$$\mathbf{Pb} = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{pmatrix} \begin{pmatrix} b_1 \\ b_2 \\ b_3 \end{pmatrix} = \begin{pmatrix} b_3 \\ b_2 \\ b_1 \end{pmatrix}$$

Definizione 3.2.10 (Diagonale dominante per righe)**.** Si dice **diagonale dominante per righe** una matrice le cui entrate rispettano la seguente proprietà

$$|a_{ii}| > \sum_{\substack{j=1 \\ j \neq i}}^{n} |a_{ij}| \quad \text{per } i = 1, \ldots, n$$

Ovvero se per ogni riga l'elemento sulla diagonale principale è in modulo maggiore della somma dei moduli degli altri elementi della riga

Definizione 3.2.11 (Diagonale dominante per colonne)**.** Si dice **diagonale dominante per righe** una matrice le cui entrate rispettano la seguente proprietà

$$|a_{jj}| > \sum_{\substack{i=1 \\ i \neq j}}^{n} |a_{ij}| \quad \text{per } j = 1, \ldots, n$$

Ovvero se per ogni colonna l'elemento sulla diagonale principale è in modulo maggiore della somma dei moduli degli altri elementi della colonna

Definizione 3.2.12 (Definita positiva)**.** Una matrice **simmetrica A** si dice **definita positiva** se

$$\mathbf{x^T A x} > 0 \quad \forall \mathbf{x} \neq 0$$

Teorema 3.2.1. La matrice $\mathbf{A} = \mathbf{B^T B}$ con $\mathbf{B} \in \mathbb{R}^{n,n}$ avente determinate non nullo è definita positiva.

Dimostrazione. **A** è simmetrica per costruzione, infatti

$$A = B^T B \quad A^T = (B^T B)^T = B^T (B^T)^T = A$$

Sia $x \neq \vec{0}$ e $y = Bx$

$$x^T A x = x^T (B^T b) x = x^T B^T B x = (x^T B^T)(Bx) = (Bx)^T (Bx) = y^T y$$

$$y^T y = \begin{bmatrix} y_1, \ldots, y_n \end{bmatrix} \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix} = y_1^2 + y_2^2 + \ldots + y_n^2 = ||y||_2^2$$

$$\Rightarrow x^T A x = ||y||_2^2 \geq 0 \quad \text{e} \quad ||y||_2^2 = 0 \leftrightarrow y = \vec{0}$$

Dal momento che $y \neq \vec{0}$ perchè essendo $y = Bx$ ed essendo $x \neq \vec{0}$ e B di rango massimo si può concludere che

$$x^T A x > 0$$

□

3.3 Matrici a blocchi

Definizione 3.3.1 (Matrice a blocchi)**.** Una **matrice a blocchi** è una matrice descritta in termini delle sue sottomatrici anziché in termini dei suoi elementi. Una descrizione di una matrice a blocchi non è unica.

$$\mathbf{A} = \left(\begin{array}{cc|c} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ \hline a_{31} & a_{32} & a_{33} \end{array}\right) = \begin{pmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{A}_{21} & \mathbf{A}_{22} \end{pmatrix}$$

La **trasposta** di una **matrice a blocchi** si ottiene nel seguente modo:

$$\mathbf{A}^T = \begin{pmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} & \cdots & \mathbf{A}_{1n} \\ \mathbf{A}_{21} & \mathbf{A}_{22} & \cdots & \mathbf{A}_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{A}_{m1} & \mathbf{A}_{m2} & \cdots & \mathbf{A}_{mn} \end{pmatrix}^T = \begin{pmatrix} \mathbf{A}_{11}^T & \mathbf{A}_{21}^T & \cdots & \mathbf{A}_{m1}^T \\ \mathbf{A}_{12}^T & \mathbf{A}_{22}^T & \cdots & \mathbf{A}_{m2}^T \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{A}_{1n}^T & \mathbf{A}_{2n}^T & \cdots & \mathbf{A}_{mn}^T \end{pmatrix}$$

Quando il numero dei blocchi e le loro dimensioni sono compatibili, le operazioni tra matrici a blocchi possono essere effettuate trattando i blocchi come scalari.

Esempio 3.3.1. Siano

$$\mathbf{A} = \begin{pmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{A}_{21} & \mathbf{A}_{22} \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} \mathbf{B}_{11} & \mathbf{B}_{12} \\ \mathbf{B}_{21} & \mathbf{B}_{22} \end{pmatrix}$$

due matrici con la stessa partizione a blocchi. Si ha allora

$$\mathbf{A} + \mathbf{B} = \begin{pmatrix} \mathbf{A}_{11} + \mathbf{B}_{11} & \mathbf{A}_{12} + \mathbf{B}_{12} \\ \mathbf{A}_{21} + \mathbf{B}_{21} & \mathbf{A}_{22} + \mathbf{B}_{22} \end{pmatrix}, \quad \alpha\mathbf{A} = \begin{pmatrix} \alpha\mathbf{A}_{11} & \alpha\mathbf{A}_{12} \\ \alpha\mathbf{A}_{21} & \alpha\mathbf{A}_{22} \end{pmatrix}$$

$$\mathbf{AB} = \begin{pmatrix} \mathbf{A}_{11}\mathbf{B}_{11} + \mathbf{A}_{12}\mathbf{B}_{21} & \mathbf{A}_{11}\mathbf{B}_{12} + \mathbf{A}_{12}\mathbf{B}_{22} \\ \mathbf{A}_{21}\mathbf{B}_{11} + \mathbf{A}_{22}\mathbf{B}_{21} & \mathbf{A}_{21}\mathbf{B}_{12} + \mathbf{A}_{22}\mathbf{B}_{22} \end{pmatrix}$$

Quest'ultimo risultato è alla base degli algoritmi di calcolo matriciale dei calcolatori dal momento che riduce il calcolo di un prodotto di matrici a somme di prodotti di matrici più piccoli. È particolarmente utile nel caso di matrici sparse (che presentano larghe aree formate da 0), perchè si individuano dei blocchi formati interamente da zeri che non inficiano nel calcolo

Definizione 3.3.2 (Matrice triangolare a blocchi)**.** Una matrice triangolare superiore (inferiore / superiore) a blocchi 'e una matrice quadrata che ha blocchi quadrati sulla diagonale e i cui blocchi sotto (sopra / sotto) la diagonale principale contengono solo zeri:

$$\mathbf{A} = \begin{pmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} & \cdots & \cdots & \mathbf{A}_{1n} \\ \mathbf{0} & \mathbf{A}_{22} & \cdots & \cdots & \mathbf{A}_{2n} \\ \vdots & \vdots & \ddots & & \vdots \\ \mathbf{0} & \mathbf{0} & \cdots & & \mathbf{A}_{nn} \end{pmatrix}$$

Per le matrici diagonali a blocchi vale il seguente risultato:

$$\det(\mathbf{A}) = \prod_{i=1}^{n} \det(\mathbf{A}_{ii})$$

3.4 Condizionamento di un sistema lineare

Consideriamo il sistema lineare di n equazioni nelle n incognite $x_1, \ldots, x_n$:

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2 \\ \qquad \vdots \qquad \vdots \qquad \ddots \qquad \vdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n = b_n \end{cases}$$

Il sistema può essere riscritto in forma matriciale e si ha:

$$\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix} \iff \mathbf{Ax} = \mathbf{b}$$

Assumiamo che il sistema abbia un' unica soluzione, in questo caso la matrice $\mathbf{A}$ si dice **non singolare** ($det(\mathbf{A}) \neq 0$), dal momento che ha rango massimo.

Andiamo a studiare il condizionamento del nostro problema numerico. Forniremo la dimostrazione del caso con $\mathbf{A}$ non perturbata, $\bar{\mathbf{b}}$ perturbato e $\bar{\mathbf{x}}$ soluzione in aritmetica esatta del sistema $\mathbf{A}\bar{\mathbf{x}} = \bar{\mathbf{b}}$.
Successivamente enunceremo senza dimostrare il caso più generale con $\bar{\mathbf{A}}$ e $\bar{\mathbf{b}}$ perturbati e $\bar{\mathbf{x}}$ soluzione in aritmetica esatta del sistema $\bar{\mathbf{A}}\bar{\mathbf{x}} = \bar{\mathbf{b}}$

Teorema 3.4.1. Dato un problema numerico del tipo $\mathbf{A}\bar{\mathbf{x}} = \bar{\mathbf{b}}$ vale la seguente relazione

$$\frac{||\mathbf{x} - \bar{\mathbf{x}}||_p}{||\mathbf{x}||_p} \leq K_p(\mathbf{A}) \frac{||\mathbf{b} - \bar{\mathbf{b}}||_p}{||\mathbf{b}||_p}$$

con $K_p(\mathbf{A}) = ||\mathbf{A}||_p ||\mathbf{A}^{-1}||_p$

Dimostrazione. Consideriamo i due problemi numerici

$$\mathbf{A}\bar{\mathbf{x}} = \bar{\mathbf{b}} \qquad \mathbf{Ax} = \mathbf{b}$$

sottraendo membro a membro otteniamo

$$\mathbf{A}(\bar{\mathbf{x}} - \mathbf{x}) = \bar{\mathbf{b}} - \mathbf{b} \Rightarrow \bar{\mathbf{x}} - \mathbf{x} = \mathbf{A}^{-1}(\bar{\mathbf{b}} - \mathbf{b}) \Rightarrow$$

$$\Rightarrow ||\bar{\mathbf{x}} - \mathbf{x}||_p = ||\mathbf{A}^{-1}(\bar{\mathbf{b}} - \mathbf{b})||_p \leq ||\mathbf{A}||_p ||\bar{\mathbf{b}} - \mathbf{b}||_p$$

$$\mathbf{Ax} = \mathbf{b} \Rightarrow ||\mathbf{b}||_p = ||\mathbf{Ax}|| \leq ||\mathbf{x}||_p ||\mathbf{A}||_p \Rightarrow ||\mathbf{x}||_p \geq \frac{||\mathbf{b}||_p}{||\mathbf{A}||_p} \Rightarrow$$

$$\Rightarrow \frac{1}{||\mathbf{x}||_p} \leq \frac{||\mathbf{A}||_p}{||\mathbf{b}||_p}$$

$$\frac{||\bar{\mathbf{x}} - \mathbf{x}||}{||\bar{\mathbf{x}}||_p} \leq ||\mathbf{A}^{-1}||_p ||\mathbf{A}||_p \frac{||\bar{\mathbf{b}} - \mathbf{b}||}{||\bar{\mathbf{b}}||_p}$$

$$K_p(\mathbf{A}) = ||\mathbf{A}^{-1}||_p ||\mathbf{A}||_p$$

□

Teorema 3.4.2. Dato un problema numerico del tipo $\bar{\mathbf{A}}\bar{\mathbf{x}} = \bar{\mathbf{b}}$.
Se $||\mathbf{A} - \bar{\mathbf{A}}||_p < \frac{1}{2||\mathbf{A}^{-1}||_p}$ vale la seguente relazione

$$\frac{\|\mathbf{x} - \bar{\mathbf{x}}\|_p}{\|\mathbf{x}\|_p} \leq 2K_p(\mathbf{A}) \left(\frac{\|\mathbf{A} - \bar{\mathbf{A}}\|_p}{\|\mathbf{A}\|_p} + \frac{\|\mathbf{b} - \bar{\mathbf{b}}\|_p}{\|\mathbf{b}\|_p} \right)$$

con

$$K_p(\mathbf{A}) = \|\mathbf{A}\|_p \, \|\mathbf{A}^{-1}\|_p$$

$K_p(\mathbf{A})$ viene definito **numero di condizionamento** del sistema $\mathbf{Ax} = \mathbf{b}$.
Si osservi che

$$K_p(\mathbf{A}) = \|\mathbf{A}\|_p \, \|\mathbf{A}^{-1}\|_p \geq \|\mathbf{AA}^{-1}\|_p = \|\mathbf{I}\|_p = 1$$

e quindi che

1. se $K(\mathbf{A}) \approx 1$, la matrice $\mathbf{A}$ si dice **ben condizionata** $\Rightarrow$ a piccole perturbazioni sui dati corrispondono pertubazioni sui risultati al più dello stesso ordine di grandezza

2. se $K(\mathbf{A}) >> 1$ la matrice $\mathbf{A}$ si dice **mal condizionata** $\Rightarrow$ a piccole perturbazioni sui dati possono corrispondere grandi perturbazioni sui risultati

Esempio 3.4.1.

$$\mathbf{D} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 10^{-10} \end{bmatrix}$$

$K_1(\mathbf{D}) = \|\mathbf{D}\|_1 \|\mathbf{D}^{-1}\|_1 = 10^{10}$. $\mathbf{D}$ è mal condizionata.

Due classiche matrici mal condizionate sono:
la **matrice di Hilbert**

$$\mathbf{H}_n = \begin{pmatrix} 1 & \frac{1}{2} & \cdots & \frac{1}{n} \\ \frac{1}{2} & \frac{1}{3} & \cdots & \frac{1}{n+1} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{1}{n} & \frac{1}{n+1} & \cdots & \frac{1}{2n-1} \end{pmatrix}$$

e la **matrice di Vandermonde**

$$\mathbf{V}_n = \begin{pmatrix} x_1^n & x_1^{n-1} & \cdots & x_1 & 1 \\ x_2^n & x_2^{n-1} & \cdots & x_2 & 1 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ x_{n+1}^n & x_{n+1}^{n-1} & \cdots & x_{n+1} & 1 \end{pmatrix}$$

.

Il condizionamento di un sistema lineare **non** dipende dalla **norma scelta**. Infatti tutte le norme hanno lo stesso ordine di grandezza.

Seguono una serie di funzioni MatLab utili per il calcolo del condizionamento di un sistema lineare e per generare le matrici di Hilbert e di Vandermonde

MATLAB
```
cond(A,1) % Calcola il numero di condizionamento in norma 1
del sistema Ax = b

cond(A, inf) % Calcola il numero di condizionamento in norma
infinito del sistema Ax = b

hilb(n) % genera la matrice di Hilbert di ordine n

vander(x) % Genera la matrice di Vandermonde associata al
vettore x con componenti x(1) , ...  , x(n+1)
```

3.5 Metodi numerici per i sistemi lineari

In questa sezione descriveremo alcuni metodi numerici e i relativi algoritmi per la risoluzione di sistemi lineari di ordine n. Per ognuno di questi forniremo anche un'espressione del loro costo computazionale in funzione dell'ordine n. Il costo verrà espresso in notazione asintotica (es $O(n^2)$) e quindi deve essere considerato valido per n molto grande

3.5.1 Metodi numerici per sistemi diagonali

Sia **A** una matrice diagonale associata al seguente sistema lineare:

$$\begin{cases} a_{11}x_1 & & & & & = & b_1 \\ & a_{22}x_2 & & & & = & b_2 \\ & & a_{33}x_3 & & & = & b_3 \\ & & & \ddots & & \vdots & \\ & & & & a_{nn}x_n & = & b_n \end{cases}$$

La soluzione del sistema si ricava eseguendo n divisioni del tipo:

$$x_i = \frac{b_i}{a_{ii}}, \qquad i = 1, \ldots, n$$

Segue un'implementazione di tale algoritmo in MatLab:

```
MATLAB d = [1 2 3 4 5 6];
n = length(d);
A = diag(d);
b = n :-1:1;
x = zeros(n,1);

for i = 1:n
x(i) = b(i) / A(i,i);
end
```

3.5.2 Metodi numerici per sistemi triangolari

Sia **A** una matrice **triangolare superiore** associata al seguente sistema lineare:

$$\begin{cases} a_{11}x_1 & + & a_{12}x_2 & + & \cdots & + & a_{1n}x_n & = & b_1 \\ & & \vdots & & \ddots & & \vdots & = & \vdots \\ & & & & a_{n-1,n-1}x_{n-1} & + & a_{n-1,n}x_n & = & b_{n-1} \\ & & & & & & a_{nn}x_n & = & b_n \end{cases}$$

Un metodo per la risoluzione di questo sistema è quello di ricavare x_n dall'ultima equazione e poi procedere per sostituzione ricorsiva fino ad arrivare alla prima equazione. Si può notare che, per la struttura della matrice **A**, ad ogni passaggio dobbiamo risolvere un'equazione in una sola incognita con la stessa struttura. Andiamo a formalizzare.
Il metodo numerico è detto **metodo di sostituzione all'indietro** e l'algoritmo che lo implementa è:

$$x_n = \frac{b_n}{a_{nn}}$$

$$x_i = \frac{b_i - \sum_{j=i+1}^{n} a_{ij}x_j}{a_{ii}} \qquad i = n-1, \ldots, 1$$

Il costo computazionale è $\frac{n^2}{2} = O(n^2)$.
Un'implementazione di tale algoritmo in MatLab è:

MATLAB

```
function x = indietro(A ,b)
n = lenght(b);
x = zeros(n,1);
% Passo base della ricorsione, risolve l'utlima equazione
x(n) = b(n) / A(n,n);
% Risale il sistema dalla riga n-1 alla riga 1
for i = n-1:-1:1
% Calcola la somma dei termini dell'i-esima equazione dal
termine i+1-esimo a quello n-esimo, sfruttando il prodotto
scalare tra due vettori
s = A(i , i+1:n) * x(i+1:n)
% Calcola il valore dell'i-esima incognita del sistema
sottraendo al termine noto della i-esima equazione (b(i))
s e poi dividendo per il coefficiente dell'i-esima incognita
nell'i-esima equazione
x(i) = (b(i) - s) / A(i,i);
end
end
```

Sia **A** una matrice **triangolare inferiore** associata al seguente sistema lineare:

$$\begin{cases} a_{11}x_1 & & & & = b_1 \\ a_{21}x_1 & +a_{22}x_2 & & & = b_2 \\ \vdots & \vdots & \ddots & & \vdots \\ a_{n1}x_1 & +a_{n2}x_2 & +\cdots & +a_{nn}x_n & = b_n \end{cases}$$

Un metodo per risolvere questo sistema è il **metodo di sostituzione in avanti**. Concettualmente è molto simile al **metodo di sostituzione all'indietro**: si ricava x_1 risolvendo la prima equazione e poi in modo ricorsivo si risolvono le equazioni successive fino alla n-esima. Anche in questo caso per la struttura della matrice **A** tutte le equazioni sono in un'unica incognita.

L'algoritmo che implementa tale metodo è il seguente:

$$x_1 = \frac{b_1}{a_{11}}$$

$$x_i = \frac{b_i - \sum_{j=1}^{i-1} a_{ij} x_j}{a_{ii}} \quad \text{per } i = 2, \ldots, n$$

Anche in questo caso il costo computazionale è $\frac{n^2}{2} = O(n^2)$.
Segue un'implementazione commentata in MatLab di tale algoritmo:

MATLAB

```
function x = avanti(A , b)
n = lenght(b);
x = zeros(n ,1) ;
% Calcola il passo base della ricorsione, risolve la prima
equazione in modo banale
x(1) = b(1) / A(1,1);
% Scende lungo il sistema dalla riga 2 alla riga n
for i = 2:n
% Calcola la somma dei termini dell'i-esima equazione
dal primo termine fino al termine i-1-esimo attraverso il
prodotto scalre di 2 vettori
s = a(i, 1:i-1) * x(1:i -1);
% Calcola il valore dell'i-esima incognita del sistema
sottraendo al termine noto della i-esima equazione (b(i))
s e poi dividendo per il coefficiente dell'i-esima incognita
nell'i-esima equazione
x(i) = (b(i) -s ) / A(i,i);
end
end
```

3.5.3 Metodo della regola di Cramer, perché no in breve

L'algebra lineare classica propone 2 metodi per la soluzione di sistemi lineari: la regola di Cramer e il metodo delle eliminazioni di Gauss.
La regola di Cramer prevede di calcolare n+1 determinanti di ordine n e ha

un costo computazionale di $(n+1)!$.
Il metodo delle eliminazioni di Gauss (descritto nel seguente paragrafo) ha un costo $n^3/3$ come costo computazionale.
Gli algoritmi con complessità fattoriale vengono risolti in tempi proibitivi anche dai più potenti calcolatori moderni.
I due algoritmi risolvono il problema con tempi che possono distare anche di decine di ordini di grandezza (es $4.6 \cdot 10^{-12}$ secondi e 492 anni). Per questa ragione la regola di Cramer risulta impraticabile.

3.5.4 Metodo delle eliminazioni di Gauss

Il metodo delle eliminazioni di Gauss prevede due fasi:

1. Trasformazione del sistema $\mathbf{Ax} = \mathbf{b}$ nel sistema $\mathbf{Ux} = \bar{\mathbf{b}}$ dove $\mathbf{U}$ è una matrice triangolare superiore

2. Risoluzione del sistema $\mathbf{Ux} = \bar{\mathbf{b}}$ tramite il metodo di sostituzione all'indietro

Il metodo si basa sul fatto che la soluzione di un sistema rimane invariata se:

1. Si scambiano due equazioni del sistema

2. Si sostituisce a un'equazione una combinazione lineare dell'equazione stessa e di un'altra

Il metodo può essere riassunto nel seguente schema di calcolo:
Per $k = 1, \ldots, n-1$,
per $i = k+1, \ldots, n$

$$\begin{cases} m_{ik} = \dfrac{a_{ik}^{(k)}}{a_{kk}^{(k)}} \\ a_{ij}^{(k+1)} = a_{ij}^{(k)} - m_{ik} a_{kj}^{(k)} \quad , \quad j = k+1, \ldots, n \\ b_i^{(k+1)} = b_i^{(k)} - m_{ik} b_k^{(k)} \end{cases}$$

$$\begin{cases} x_n = \dfrac{b_n^{(n)}}{a_{nn}^{(n)}} \\ x_k = \left(b_k^{(k)} - \displaystyle\sum_{j=k+1}^{n} a_{kj}^{(k)} x_j \right) \Big/ a_{kk}^{(k)} \quad , \quad k = n-1, \ldots, 1 \end{cases}$$

Analizziamo la prima parte.
Il primo ciclo (k) è eseguito $n-1$ volte. Questo ciclo elimina la k-esima colonna dalla riga $k+1$ fino alla riga n, attraverso il secondo ciclo (i).
Nello specifico per ogni riga (i) vengono calcolati:

1. il parametro m_{ik} come il rapporto $a_{ik}^{(k)}/a_{kk}^{k}$ tra il coefficiente di x_k dell' i-esima riga e il coefficiente di x_k della k-esima riga.

2. i nuovi coefficienti dell'i-esima riga dalla coefficiente di x_{k+1} al coefficiente di x_n (j), come $a_{ij}^{(k+1)} = a_{ij}^{(k)} - m_{ik}a_{kj}^{(k)} \quad , \quad j = k+1, \ldots, n$

3. il nuovo termine noto della i-esima riga come $b_i^{(k+1)} = b_i^{(k)} - m_{ik}b_k^{(k)}$

Si noti che ad ogni iterazione del ciclo più esterno (k) andiamo a lavorare su una sotto-matrice sempre più piccola le cui entrate sono contrassegnate con l'apice (k).

La seconda parte è essenzialmente l'algoritmo di sostituzione all'indietro che è stato già discusso precedentemente.

Come già osservato nel precedente paragrafo il costo computazionale di tale algoritmo è $n^3/3$. Se però **A** è simmetrica e non si effettuano scambi si dimostra che tutte le sotto-matrici contrassegnata dall' apice (k) che si vengono a formare durante la prima parte dell'algoritmo sono simmetriche anch'esse:

$$\begin{pmatrix} a_{11}^{(1)} & a_{12}^{(1)} & \cdots & a_{1k}^{(1)} & \cdots & a_{1n}^{(1)} \\ & a_{22}^{(2)} & & & & \\ & & \cdots & a_{2k}^{(2)} & \cdots & a_{2n}^{(2)} \\ & & \ddots & \vdots & & \vdots \\ & & & a_{kk}^{(k)} & \cdots & a_{kn}^{(k)} \\ & & & \vdots & \ddots & \vdots \\ & & & a_{nk}^{(k)} & \cdots & a_{nn}^{(k)} \end{pmatrix}$$

Quindi ad ogni passo k è possibile generare solo la metà degli elementi della sotto-matrice (k+1) dal momento che l'altra metà è speculare. Il tal caso il costo computazionale diventa $n^3/6$.

Osserviamo che per ottimizzare memoria è possibile lavorare esclusivamente sulla matrice **A** e sul vettore **b**:

- Ad ogni passo k gli elementi $a_{ij}^{(k+1)}$ e $b_i^{(k+1)}$ possono essere memorizzati nelle posizioni ij di **A** e i in **b**, dove erano memorizzati $a_{ij}^{(k)}$ e $b_i^{(k)}$

- i parametri m_{ik} possono essere memorizzati nella matrice **A** nella posizione ik essendo $a_{ik}^{(k+1)} = 0$ per definizione

L'algoritmo che abbiamo presentato fino ad ora richiede che $a_{kk}^{(k)} \neq 0 \quad \forall k$. In caso contrario esiste ad ogni passo k una riga per la quale l'elemento della colonna k non è nullo. Possiamo allora scambiare la riga k con questa riga che abbiamo individuato.
Non è necessario effettuare scambi se:

- **A** è a diagonale dominante per righe

- **A** è a diagonale dominante per colonne

- **A** è simmetrica e definita positiva

Presentiamo un'implementazione dell'algoritmo delle eliminazioni gaussiani in MatLab nell'ipotesi che **A** rientri in uno dei 3 precedenti casi, e quindi non è necessario effettuare scambi.

MATLAB

```
function x = gaussNoscambi(A,b)
n = length(b);
% Elimina le entrate di A corrispondenti alle colonne dalla
1 alla n-1 ( indice k) e alle righe dalla k+1 alla n
for k = 1:  n-1
% Cicla attraverso le righe dalla k+1 alla n, eliminando la
colonna k.
for i = k+1:  n
% Genera il coefficiente m(i,k) e lo salva in A(i,k)
A(i,k) = A(i,k) / A(k,k);
% Applica la combinazione lineare a tutte le colonne della
riga i dalla k+1 alla n
for j = k+1:n
A(i,j) = A(i,j) - A(i,k) * A(k,j);
end
% Applica la combinazione lineare al termine noto della riga
i
b(i) = b(i) - A(i,k) * b(k);
end
end

% Algoritmo di sostituzione all'indietro
x = zeros(n,1);
x(n) = b(n) / A(n.n);
for i = n-1:-1:1
x(i) = (b(i)-A(i,i+1:n)*x(i+1:n)) / A(i,i);
end
```

3.5.5 Fattorizzazione PA=LU

Un 'altro vantaggio di salvare i moltiplicatori m_{ik} nella matrice $\mathbf{A}$ è quello di poter effettuare una fattorizzazione sulla matrice di partenza, la fattorizzazione $\mathbf{A} = \mathbf{LU}$. È possibile generare $\mathbf{L}$ e $\mathbf{U}$ alla fine del metodo delle eliminazioni gaussiane.

Infatti **L** è una matrice triangolare inferiore i cui elementi sono definita da

$$l_{ii} = 1, \quad l_{ij} = a_{ij} \quad \text{per} \quad i > j$$

ovvero la matrice dei moltiplicatori m_{ik} con diagonale unitaria
e **U** è una matrice triangolare superiore i cui elementi sono

$$u_{ij} = a_{ij} \quad \text{per} \quad j \geq i$$

ovvero la matrice del sistema equivalente al sistema di partenza.

L'implementazione in MatLab di tale fattorizzazione è nella prima parte uguale al metodo delle eliminazioni di Gauss ma al posto dell'algoritmo di sostituzione all'indietro c'è il seguente codice

MATLAB
```
% tril restituisce gli elementi che si trovano sulla
diagonale -1 di A e sotto di essa.
L = tril(A,-1) + eye(n);
% triu restituisce gli elementi che si trovano nella e sopra
la diagonale principale di A
U = triu(A);
```

In caso di scambi di righe si realizza la seguente fattorizzazione:

$$\mathbf{PA} = \mathbf{LU}$$

dove **P** è la matrice di permutazione che tiene traccia degli scambi di riga effettuati durante il metodo di Gauss.
Il costo computazionale di tali fattorizzazioni è lo stesso del metodo, $n^3/3$.
Per garantire una migliore stabilità numerica se a_{kk}^k pur non essendo 0 è molto piccolo si effettua comunque uno scambio di riga, individuando la riga nel seguente modo:

$$\left|a_{r_k}^{(k)}\right| = \max_{k \leq i \leq n} \left|a_{i_k}^{(k)}\right|$$

ovvero trovando il massimo tra nella colonna k dalla riga k a quella n.
Tale strategia è chiamata **pivoting parziale**.

Il pivoting è superfluo quando:

- **A** è a diagonale dominante per colonne
- **A** è simmetrica definita positiva

In MatLab si possono utilizzare le funzioni per il metodo di Gauss e per la fattorizzazione PA = LU già integrate

MATLAB
```
% Calcola la soluzione di Ax = b con il metodo di Gauss con
pivoting parziale.  A seconda delle caratteristiche di A
sceglie un metodo più adatto.  Se A non rientra in nessuna
categoria speciale utile viene effettuata una classica
fattorizzazione PA = LU
x = A \b;
% Calcolo del residuo
r = A * x - b
% Calcola i fattori L , U , P della fattorizzazione PA = LU
[ L,U,P ] = lu(A);
```

3.5.6 Applicazioni della fattorizzazione PA = LU

Esempio 3.5.1 (Applicazione 1. Risoluzione del sistema lineare Ax = b)**.** Se si conoscono i fattori P , L ,U si può procedere in questo modo

$$\mathbf{Ax} = \mathbf{b} \Rightarrow \mathbf{PAx} = \mathbf{Pb} \Rightarrow \mathbf{LUx} = \mathbf{Pb}$$

da cui segue la risoluzione di 2 sistemi triangolari

$$\begin{cases} Ly = Pb & \Rightarrow \quad y \\ Ux = y & \Rightarrow \quad x \end{cases}$$

Il costo computazione scende così a n^2.
Ovviamente se si devono calcolare U , L e P il costo rimane di $n^3/3$.

Esempio 3.5.2 (Applicazione 2. Risoluzione di più sistemi lineari $x_i = b_i$)**.** Per risolvere p sistemi lineari aventi tutti la stessa matrice **A** è possibile calcolare la fattorizzazione **PA** = **LU** e poi risolvere 2p sistemi triangolari. Il costo di queste operazioni è $n^3/3 + pn^2$ dove se p è molto più piccolo di n

il costo rimane circa $n^3/3$.
La risoluzione di p sistemi tramite il metodo delle eliminazioni di Gauss costerebbe $pn^3/3$

Esempio 3.5.3 (Applicazione 3. Calcolo del determinante di una matrice)**.** Per calcolare il determinante di $\mathbf{A}$ si può procedere nel seguente modo

$$\det(\mathbf{PA}) = \det(\mathbf{LU}) \Longrightarrow \det(\mathbf{P})\det(\mathbf{A}) = \det(\mathbf{L})\det(\mathbf{U})$$

Dal momento che $det(\mathbf{L}) = 1$ e $det(\mathbf{U}) = \prod_{i=1}^{n} u_{ii}$ e $det(\mathbf{P}) = (-1)^s$ dove s è il numero di scambi effettuati allora

$$det(\mathbf{A}) = (-1)^s \prod_{i=n}^{n} u_{ii}$$

Questa procedura sta alla base del domando det(A) di MatLab.

Esempio 3.5.4 (Applicazione 4. Calcolo dell'inversa di una matrice)**.** Un metodo efficiente per il calcolo dell'inversa di una matrice è il seguente:

$$\mathbf{PA} = \mathbf{LU} \Longrightarrow (\mathbf{PA})^{-1} = (\mathbf{LU})^{-1} \Longrightarrow \mathbf{A}^{-1}\mathbf{P}^{-1} = \mathbf{U}^{-1}\mathbf{L}^{-1}$$

da cui

$$\mathbf{A}^{-1} = \mathbf{U}^{-1}\mathbf{L}^{-1}\mathbf{P}$$

Il costo computazionale di tale procedura è n^3.
Tale procedura è alla base del comando inv(A) in MatLab.

3.5.7 Fattorizzazione di Choleski

Se $\mathbf{A}$ è una matrice simmetrica definita positiva si può effettuare la seguente fattorizzazione

$$\mathbf{A} = \mathbf{R^T R}$$

dove $\mathbf{R}$ è una matrice triangolare superiore con elementi positivi sulla diagonale principale.
tale fattorizzazione è detta **fattorizzazione di Choleski**.
Le applicazioni sono simili a quella della fattorizzazione $\mathbf{PA} = \mathbf{LU}$. Il vantaggio è che il costo computazionale è $n^3/6$ per la fattorizzazione e di $2n^2/3$ per il calcolo dell'inversa.

MATLAB
```
% Produce il termine R della fattorizzazione di Choleski
R = chol(A);
```

3.5.8 Fattorizzazione QR

Teorema 3.5.1. Ogni matrice $\mathbf{A} \in \mathbb{R}^{m,n}$ è fattorizzabile nella forma

$$\mathbf{A} = \mathbf{QR}$$

dove $\mathbf{Q} \in \mathbb{R}^{m,m}$ **ortogonale** e $\mathbf{R} \in \mathbb{R}^{m,n}$ con elementi $r_{ij} = 0 \quad \forall i > j$.

Essendo $\mathbf{Q}$ ortogonale ha tutti i vettori colonna a due a due ortogonali. Per $m < n$ e per $m > n$ si ha

$$\mathbf{R} = \begin{pmatrix} r_{11} & r_{12} & \cdots & \cdots & \cdots & r_{1n} \\ 0 & r_{22} & \cdots & \cdots & \cdots & r_{2n} \\ \vdots & \vdots & \ddots & & & \vdots \\ 0 & 0 & \cdots & r_{mm} & \cdots & r_{mn} \end{pmatrix}, \quad \mathbf{R} = \begin{pmatrix} r_{11} & r_{12} & \cdots & r_{1n} \\ 0 & r_{22} & \cdots & r_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & r_{nn} \\ 0 & 0 & \cdots & 0 \\ \vdots & \vdots & & \vdots \\ 0 & 0 & \cdots & 0 \end{pmatrix}$$

In generale la fattorizzazione QR di una matrice $\mathbf{A}$ non è unica.

Teorema 3.5.2. Sia $\mathbf{A} \in \mathbb{R}^{m,n}$, $m \geq n$, di rango massimo $\mathbf{n}$. Allora la matrice $\mathbf{R}$ della fattorizzazione QR si scrive come:

$$\mathbf{R} = \begin{pmatrix} \tilde{\mathbf{R}} \\ \mathbf{O} \end{pmatrix}$$

con $\mathbf{O}$ matrice nulla e $\tilde{\mathbf{R}}$ triangolare superiore non singolare.
Vale la fattorizzazione

$$\mathbf{A} = \tilde{\mathbf{Q}}\tilde{\mathbf{R}}$$

dove $\tilde{\mathbf{Q}} \in \mathbb{R}^{m,n}$, formata da vettori colonna ortonormali, è la sotto-matrice di $\mathbf{Q}$ ottenuta considerando le prime $\mathbf{n}$ colonne. Tale fattorizzazione risulta unicamente determinata.

In MatLab una fattorizzazione QR può essere ottenuta tramite la funzione `qr()` dal costo computazionale $O(n^3)$

MATLAB
```
[Q,R] = qr(A)
```

Esempio 3.5.5 (Applicazione 1. **Risoluzione di un sistema lineare**). Come le altre fattorizzazioni la fattorizzazione QR può essere utilizzata per la risoluzione di un sistema lineare.
Infatti dato un sistema $\mathbf{Ax} = \mathbf{b}$ se si conosce una fattorizzazione QR per la matrice $\mathbf{A}$ si può procedere nel seguente modo

$$\mathbf{Ax} = \mathbf{b} \Rightarrow \mathbf{QRx} = \mathbf{b} \Rightarrow \mathbf{Rx} = \mathbf{Q^T b}$$

Essendo $\mathbf{R}$ triangolare superiore il costo computazionale è quella di una sostituzione all'indietro ovvero $O\left(\frac{n^2}{2}\right)$.
Se però non si possiede già una fattorizzazione QR non conviene usare tale procedura in quando il costo computazionale sarebbe di $O\left(n^3 + \frac{n^2}{2}\right) = O\left(n^3\right)$, più oneroso di $O\left(\frac{n^3}{3}\right)$ per il metodo delle eliminazioni di Gauss.

3.5.9 Problema dei minimi quadrati (applicazione QR)

La fattorizzazione QR è utile nella risoluzione di un sistema lineare sovradeterminato.
Il metodo che illustreremo per trovare una "soluzione" di tale sistema è detto metodo dei minimi quadrati.
Partiamo con una serie di definizioni utili per il processo.

Definizione 3.5.1. Consideriamo il sistema

$$\mathbf{Ax} = \mathbf{b}, \quad \mathbf{A} \in \mathbb{R}^{m,n}, \quad \mathbf{x} \in \mathbb{R}^n, \quad \mathbf{b} \in \mathbb{R}^m, \quad m > n$$

tale sistema si dice **sovradeterminato** dal momento che il delle equazioni lineari m è maggiore del numero di incognite n.
Un sistema sovradeterminato può non ammettere soluzioni in senso classico,

indipendentemente dal rango.
Infatti se consideriamo l'applicazione lineare $L_A : \mathbb{R}^n \to \mathbb{R}^m, x \mapsto Ax$ e applichiamo il teorema di *rank-nullity* otteniamo

$$n = \dim(\mathrm{Ker}(L_A)) + \dim(\mathrm{Im}(L_A)) \Rightarrow \dim(\mathrm{Im}(L_A)) = \mathrm{rk}(A) \leq n$$

ovvero che il rango di A è controllato dall'alto dalla dimensione del dominio. Questo è equivalente a dire che l'immagine di L_A è in ogni caso un sottospazio proprio di $\mathbb{R}^m$ (al massimo un sottospazio di dimensione $n < m$) e quindi possiamo trovare sempre un vettore (in realtà infiniti) $b \in \mathbb{R}^m, b \notin Im(L_A)$ ovvero tale che il sistema $Ax = b$ non ammette soluzioni.
Se il sistema non ammette soluzioni in senso classico si ricercano i vettori $\mathbf{x}$ per i quali il vettore **residuo** $\mathbf{Ax} - \mathbf{b}$ è il più piccolo possibile (norma minima). Se il vettore **residuo** è 0 allora $\mathbf{x}$ è una soluzione in senso classico del sistema.
Osserviamo che stiamo cercando la proiezione ortogonale di $\mathbf{b}$ sul sottospazio $Im(L_A) = \{\mathbf{Ax} : \mathbf{x} \in \mathbb{R}^n\}$

Definizione 3.5.2. Si dice che il vettore $\mathbf{x}$ è **soluzione del sistema sovradeterminato** $\mathbf{Ax} = \mathbf{b}$ rispetto alla norma $\|\cdot\|_p$ se

$$\min_{\mathbf{y} \in \mathbb{R}^n} \|\mathbf{Ay} - \mathbf{b}\| = \|\mathbf{Ax} - \mathbf{b}\|$$

ovvero se $\mathbf{Ax}$ è la proiezione ortogonale di $\mathbf{b}$ su $Im(L_A)$.
Se scegliamo come norma quella euclidea $\|\cdot\|_2$ il problema

$$\min_{\mathbf{y} \in \mathbb{R}^n} \|\mathbf{Ay} - \mathbf{b}\|_2 = \|\mathbf{Ax} - \mathbf{b}\|_2$$

si dice **problema dei minimi quadrati**, e un vettore $\mathbf{x}$ che lo risolve si chiama **soluzione nel senso dei minimi quadrati**.

Il seguente teorema da una caratterizzazione all'insieme X delle soluzioni $\mathbf{x} \in \mathbb{R}^n$ di una problema dei minimi quadrati.

Teorema 3.5.3. Il problema dei minimi quadrati ammette sempre soluzione ovvero $X \neq \emptyset$.
Inoltre

- $\mathbf{x} \in X$ se e solo se
$$\mathbf{A^T Ax} = \mathbf{A^T b}$$
tale sistema è detto **sistema delle equazioni normali**

- $|X| = 1 \Leftrightarrow rank(A) = n$
- In ogni caso esiste uno e un solo vettore tale che

$$\|\mathbf{x}^\star\|_2 = \min_{\mathbf{x}\in X} \|\mathbf{x}\|_2$$

il vettore $\mathbf{x}^*$ è detto **soluzione di minima norma**

Osserviamo che la proiezione ortogonale di $\mathbf{b}$ su $Im(L_A)$ è sempre unica. Se però $rk(A) = n$ allora la restrizione di codominio di L_A alla sua immagine è un'isomorfismo e il vettore $\mathbf{x} \in \mathbb{R}^n$ tale che $p(\mathbf{b}) = \mathbf{Ax}$ è anch'esso unico. Questo giustifica il punto 2 del teorema.

Torniamo alla risoluzione di un sistema sovradeterminato.
Come segue naturalmente dal teorema 3.5.3 distinguiamo il caso in cui $\mathbf{A}$ ha rango massimo e quello in cui non ha rango massimo.

1) $\mathbf{A}$ ha rango massimo
Potremmo trovare l'unica soluzione del problema risolvendo il sistema delle equazioni normali:

$$\mathbf{A^T A x} = \mathbf{A^T b}$$

Si potrebbe pensare di risolvere il sistema usando la fattorizzazione di Choleski dal momento che $\mathbf{A^T A}$ è simmetrica definita positiva, avendo $\mathbf{A}$ rango massimo.
Tale procedura è però fortemente sconsigliata essendo il sistema delle equazioni normali fortemente mal condizionato.

Un metodo stabile per la risoluzione del problema è per l'appunto basata sulla fattorizzazione $\mathbf{A{=}QR}$.
Introduciamo prima il seguente lemma tecnico

Lemma 3.5.1 (lemma:matrice-norma-2)**.** Data una matrice $\mathbf{B}$ ortogonale è sempre vero che

$$\left\|\mathbf{B}^\top \mathbf{z}\right\|_2^2 = \left(\mathbf{B}^\top \mathbf{z}\right)^\top \mathbf{B}^\top \mathbf{z} = \mathbf{z}^\top \mathbf{B}\mathbf{B}^\top \mathbf{z} = \|\mathbf{z}\|_2^2,$$

Dal momento che stiamo effettuando un problema di ricerca di un minimo su una quantità sempre positiva (norma) possiamo cercare un minimo del quadrato di tale quantità.

$$\|\mathbf{Ay} - \mathbf{b}\|_2^2 = \left\|\mathbf{Q}^\top(\mathbf{Ay} - \mathbf{b})\right\|_2^2 = \left\|\mathbf{Ry} - \mathbf{Q}^\top \mathbf{b}\right\|_2^2 = \|\mathbf{Ry} - \mathbf{c}\|_2^2$$

con $\mathbf{c} := \mathbf{Q}^\top \mathbf{b}$.

Procediamo a minimizzare $\|\mathbf{R}\mathbf{y} - \mathbf{c}\|_2^2$.
Poichè $\mathbf{A}$ ha rango massimo

$$\mathbf{R} = \begin{pmatrix} \tilde{\mathbf{R}} \\ \mathbf{O} \end{pmatrix}, \quad \tilde{\mathbf{R}} \in \mathbb{R}^{n,n}, \quad \mathbf{O} \in \mathbb{R}^{m-n,n}$$

con $\tilde{\mathbf{R}}$ triangolare superiore e non singolare e $\mathbf{O}$ matrice nulla.
Partizioniamo $\mathbf{c}$ come segue:

$$\mathbf{c} = \begin{pmatrix} \mathbf{c}_1 \\ \mathbf{c}_2 \end{pmatrix}, \quad \mathbf{c}_1 \in \mathbb{R}^n, \quad \mathbf{c}_2 \in \mathbb{R}^{m-n}$$

ne segue che

$$\|\mathbf{R}\mathbf{y} - \mathbf{c}\|_2^2 = \left\| \begin{pmatrix} \tilde{\mathbf{R}}\mathbf{y} \\ \mathbf{0} \end{pmatrix} - \begin{pmatrix} \mathbf{c}_1 \\ \mathbf{c}_2 \end{pmatrix} \right\|_2^2 = \left\| \begin{pmatrix} \tilde{\mathbf{R}}\mathbf{y} - \mathbf{c}_1 \\ -\mathbf{c}_2 \end{pmatrix} \right\|_2^2 = \left\| \tilde{\mathbf{R}}\mathbf{y} - \mathbf{c}_1 \right\|_2^2 + \|\mathbf{c}_2\|_2^2$$

Essendo $\tilde{\mathbf{R}}$ non singolare il sistema $\tilde{\mathbf{R}}\mathbf{y} = \mathbf{c_1}$ ammette soluzione unica $\mathbf{x}^*$ per la quale si ha $\left\| \tilde{\mathbf{R}}\mathbf{x}^* - \mathbf{c}_1 \right\|_2^2 = 0$.
Ne segue che

$$\min_{\mathbf{y} \in \mathbb{R}^n} \|\mathbf{A}\mathbf{y} - \mathbf{b}\|_2 = \|\mathbf{A}\mathbf{x}^\star - \mathbf{b}\|_2 = \|\mathbf{c}_2\|_2$$

Se $\mathbf{c_2} = 0$ allora $\mathbf{x}^*$ è una soluzione in senso classico del sistema, altrimenti è soluzione nel senso dei minimi quadrati. In tal senso la norma $\|\mathbf{c}_2\|_2^2$ fornisce una misura del residuo.
Il costo computazionale è ovviamente pari a quello della fattorizzazione QR ovvero $O(n^3)$.

2) $\mathbf{A}$ non ha rango massimo

Se $\mathbf{A}$ non ha rango massimo allora il problema dei minimi quadrati ammette infinite soluzioni. Infatti se $\mathbf{x}$ è una soluzione particolare del problema allora anche $\mathbf{x} + \mathbf{z}$ dove $\mathbf{z}$ è soluzione del sistema omogeneo $\mathbf{A}\mathbf{z} = \mathbf{0}$ è soluzione del problema.
Inoltre se provassimo a fattorizzare una matrice $\mathbf{A}$ non di rango massimo

con la fattorizzazione QR otterremmo una matrice $\tilde{\mathbf{R}}$ non singolare se consideriamo la sottomatrice n x n (ha almeno una riga di zeri) oppure non quadrata se consideriamo la sottomatrice r x n. In entrambi i casi il sistema non ammetterebbe unica soluzione.
Potremmo vedere il problema anche da un punto di vista di applicazioni lineari.
Consideriamo l'applicazione $f : \mathbb{R}^n \to \mathbb{R}^n \quad \mathbf{y} \to \tilde{\mathbf{R}}\mathbf{y}$
Se $\mathbf{A}$ ha rango n allora $\tilde{\mathbf{R}}$ ha rango n e per il teorema del rango e della nullità

$$dim(\mathbb{R}^n) = n = dim(ker(f)) + dim(Im(f)) = dim(ker(f)) + rk(\tilde{\mathbf{R}}) =$$

$$= dim(ker(f)) + n \Rightarrow dim(ker(f)) = 0$$

Se invece $\mathbf{A}$ ha rango $r < n$ allora $\tilde{\mathbf{R}}$ ha rango r e sempre per il teorema del rango e della nullità

$$dim(\mathbb{R}^n) = n = dim(ker(f)) + dim(Im(f)) = dim(ker(f)) + rk(\tilde{\mathbf{R}}) =$$

$$= dim(ker(f)) + r \Rightarrow dim(ker(f)) = n - r \neq 0$$

Dal momento che la controimmagine di un vettore $\mathbf{b}$ è

$$f^{-1}(b) = \{x\} + ker(f)$$

per una $x|f(x) = b$, allora la controimmagine può essere "generata" da un numero di vettori pari alla dimensione di $ker(f)$ e quindi ha variabili libere pari alla dimensione di $ker(f)$.
ATTENZIONE! la controimmagine di un vettore non è quasi mai un sottospazio (l'unica eccezione è proprio ker(f)) quindi per "generare" non si intende in senso proprio di uno spazio vettoriale finitamente generato.
La soluzione è quindi unica solo quando $dim(ker(f)) = 0$ e quindi quando $\mathbf{A}$ ha rango massimo.
Potrebbe inoltre capitare che per errori numerici la matrice $\tilde{\mathbf{R}}$ abbia rango massimo con alcune entrate prossime alla zero. In questo caso il sistema ammetterebbe unica soluzione ma questa soluzione è ovviamente "sporcizia" in quanto frutto di errori di calcolo in aritmetica finita.

Il teorema 3.5.3 ci garantisce però che esiste una soluzione di norma minima ovvero tale che

$$\|\mathbf{A}\mathbf{x}^\star - \mathbf{b}\|_2 = \min_{\mathbf{y}\in\mathbb{R}^n} \|\mathbf{A}\mathbf{y} - \mathbf{b}\|_2 \quad \text{e} \quad \|\mathbf{x}^\star\|_2 = \min_{\mathbf{x}\in X} \|\mathbf{x}\|_2$$

Per calcolare tale soluzione è necessario conoscere la decomposizione ai valori singolari di **A** che tratteremo in seguito. (vedi sezione ..)

MATLAB

```
x = A  b % Se il sistema è sovradeterminato e A è una
matrice rettangolare mxn di rango massimo fornisce l'unica
soluzione nel senso dei minimi quadrati.  Se A non ha rango
massimo il comando restituisce la soluzione nel senso dei
minimi quadrati con maggior numero di 0
```

3.5.10 Approssimazione di dati con polinomi di regressione (applicazione QR)

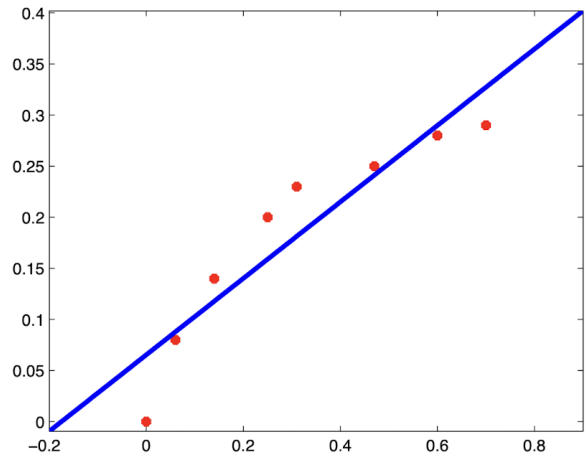
Un'altra applicazione della fattorizzazione QR e riconducibile sempre alla risoluzione di sistemi sovradeterminati è quella dell'approssimazione di m dati con un polinomio di grado $n-1$.
Consideriamo il caso $n = 2$. I risultati ottenuti valgono per ogni $n \geq 2$.
Supponiamo di avere m dati formati dalle coppie (x_i, y_i) con $i = 1, \cdots, m$.
Sia $p(x) = c_1 x + c_2$ l'equazione di una retta e imponiamo le condizioni

$$p(x_i) = c_1 x_i + c_2 = y_i$$

ottenendo il sistema sovradeterminato **Ac=b**

$$\begin{pmatrix} x_1 & 1 \\ x_2 & 1 \\ \vdots & \vdots \\ x_m & 1 \end{pmatrix} \begin{pmatrix} c_1 \\ c_2 \end{pmatrix} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{pmatrix}$$

La soluzione di tale sistema nel senso dei minimi quadrati definisce la retta (o il polinomio se $n > 2$) che meglio approssima i dati (quella con distanza minima da tutti i punti). Tale retta (o polinomio) è detta retta (o polinomio) di regressione.
La retta potrebbe non passare (e spesso è cosi) in nessun punto tra quelli iniziali.



Se la retta invece passa per tutti i punti iniziale la soluzione **c** del sistema è una soluzione in senso classico

Consideriamo il seguente esempio in MatLab

MATLAB

```
x = [0 0.06 0.14 0.25 0.31 0.47 0.60 0.70];
y = [0 .08 0.14 0.20 0.23 0.25 0.28 0.29];
b = y';
m = length(x);
A = [x' ones(n,1)];
r = rank(A); % Il rango è massimo
[m , n] = size(A);
[Q,R] = qr(A);
c = Q'*b;
c1 = c(1:n);
[a , b] = R(1:n , 1:n) \c1; % Retta y = ax+b
```

Capitolo 4

Valori singolari e scomposizione SVD

In questo capitolo tratteremo la decomposizione ai valori singolari (SVD) per una generica matrice. Complementi a questo capitolo seguiranno nel capitolo 6 dopo aver studiato gli autovalori nel capitolo 5.

4.1 Decomposizione SVD

Teorema 4.1.1. Ogni matrice $\mathbf{A} \in \mathbb{R}^{m,n}$ è fattorizzabile nella forma

$$\mathbf{A} = \mathbf{U}\mathbf{S}\mathbf{V}^{\mathbf{T}}$$

con

$$\mathbf{S} = \mathrm{diag}(s_1, \ldots, s_p) \in \mathbb{R}^{m,n}, \quad p = \min\{m, n\}, \quad s_1 \geq s_2 \geq \cdots \geq s_p \geq 0$$

e con $\mathbf{U} \in \mathbb{R}^{m,m}$ e $\mathbf{V} \in \mathbb{R}^{n,n}$ matrici ortogonali.
La matrice $\mathbf{S}$ è univocamente determinata mentre $\mathbf{U}$ e $\mathbf{V}$ non lo sono

La fattorizzazione definita sopra si dice **decomposizione ai valori singolari** della matrice $\mathbf{A}$ e i valori s_i sono detti per l'appunto valori singolari di $\mathbf{A}$.

Definizione 4.1.1. I vettori colonna $\mathbf{v_i}$ per $i = 1 \ldots n$ di $\mathbf{V}$ e $\mathbf{u_i}$ per $i = 1 \ldots m$ di $\mathbf{U}$ sono definiti rispettivamente **vettori singolari destri** e **vettori singolari sinistri** della matrice $\mathbf{A}$ dal momento che la scomposizione

precedente può essere scritta anche come

$$\mathbf{U^T A V} = \mathbf{S}$$

essendo $\mathbf{U}$ e $\mathbf{V}$ ortogonali.

La matrice $\mathbf{S}$ può avere le seguenti 3 forme

$$\begin{pmatrix} s_1 & 0 & \cdots & \cdots & 0 & 0 & 0 \\ 0 & s_2 & \cdots & \cdots & 0 & 0 & 0 \\ \vdots & \vdots & \ddots & & \vdots & 0 & 0 \\ 0 & 0 & \cdots & \cdots & s_m & 0 & 0 \end{pmatrix} \qquad \begin{pmatrix} s_1 & 0 & \cdots & \cdots & 0 \\ 0 & s_2 & \cdots & \cdots & 0 \\ \vdots & \vdots & \ddots & & \vdots \\ \vdots & \vdots & & s_n & \vdots \\ 0 & 0 & \cdots & \cdots & 0 \\ 0 & 0 & \cdots & \cdots & 0 \end{pmatrix}$$

$$\text{se } p = m \ (m < n) \qquad\qquad \text{se } p = n \ (m > n)$$

$$\operatorname{diag}(s_1, \ldots, s_n) \in \mathbb{R}^{n,n}$$
$$\text{se } p = m = n$$

Un'importantissima conseguenza della SVD è che vale sempre la relazione

$$\mathbf{USV}^T = \sum_{i=1}^{p} s_i \, \mathbf{u}_i \mathbf{v}_i^T, \quad \text{con } p = \min\{m, n\}$$

dove $\mathbf{u}_i$ e $\mathbf{v}_i^T$ sono l'i-esimo vettore singolare sinistro e l'i-esimo vettore singolare destro.
La SVD non è solo una fattorizzazione ma anche una scomposizione in addendi di una matrice $\mathbf{A}$, risultato che risulta utilissimo nelle applicazioni. In MatLab valori singolari e scomposizione SVD possono essere ottenuti nel seguente modo

MATLAB

```
d = svd(A) % resitutisce un vettore contenente in ordine
decrescente i valori singolari di A
[U,S,V] = svd(A) % Restituisce la matrice diagonale S con i
valori singolari sulla diagonale in ordine decrescente e le
due matrici U e V della scomposizione SVD
```

Il costo computazionale della scomposizione SVD tramite `svd` è $O(32/3n^3)$. Il calcolo dei valori singolari è sempre un problema ben condizionato infatti si può dimostrare il teorema

Teorema 4.1.2. Sia $\mathbf{A} \in \mathbb{R}^{m,n}$ e $\tilde{\mathbf{A}}$ una sua perturbazione e siano s_i e $\tilde{s}_i$ due loro corrispettivi valori singolari. Si ha che

$$|s_i - \tilde{s}_i| \leq \|\mathbf{A} - \tilde{\mathbf{A}}\|_2, \quad i = 1 \dots p$$

con $p = min\{m, n\}$

Passiamo ad enunciare il seguente risultato utile nelle applicazioni.

Teorema 4.1.3. Sia $\mathbf{A} \in \mathbb{R}^{m,n}$ e sia

$$\mathbf{A} = \mathbf{U}\mathbf{S}\mathbf{V}^T$$

la sua decomposizione ai valori singolari. Se risulta

$$s_1 \geq s_2 \geq \cdots \geq s_r > s_{r+1} = \cdots = s_p = 0$$

allora

1. Il rango di $\mathbf{A}$ è r.
2. $\mathbf{A} = \mathbf{U}_r\mathbf{S}_r\mathbf{V}_r^T = \sum_{i=1}^r s_i\mathbf{u}_i\mathbf{v}_i^T$ ove
   - $\mathbf{U}_r \in \mathbb{R}^{m,r}$ è la matrice le cui colonne sono $\mathbf{u}_1, \dots, \mathbf{u}_r$,
   - $\mathbf{V}_r \in \mathbb{R}^{n,r}$ è la matrice le cui colonne sono $\mathbf{v}_1, \dots, \mathbf{v}_r$,
   - $\mathbf{S}_r \in \mathbb{R}^{r,r}$ è la matrice diagonale i cui elementi diagonali sono $s_1, \dots, s_r$.
   - Le colonne $\mathbf{u}_1, \dots, \mathbf{u}_r$ di $\mathbf{U}$ formano una base ortonormale di $Im(\mathbf{A}) = \{\mathbf{A}\mathbf{x} : \mathbf{x} \in \mathbb{R}^n\}$
   - Le colonne $\mathbf{v}_1, \dots, \mathbf{v}_r$ di $\mathbf{V}$ formano una base ortonormale di $Ker(\mathbf{A}) = \{\mathbf{x} \in \mathbb{R}^n : \mathbf{A}\mathbf{x} = 0\}$
   - Le colonne $\mathbf{u}_{r+1}, \dots, \mathbf{u}_m$ di $\mathbf{U}$ formano una base ortonormale di $Ker(\mathbf{A^t})$
   - Le colonne $\mathbf{v}_1, \dots, \mathbf{v}_r$ di $\mathbf{U}$ formano una base ortonormale di $Im(\mathbf{A^t})$

La scomposizione SVD è utile per determinare la distanza tra una matrice $\mathbf{A}$ e l'insieme delle matrici con stessa dimensione di $\mathbf{A}$ e rango k. Chiariamo meglio il concetto attraverso il seguente teorema

Teorema 4.1.4. Sia $\mathbf{A} \in \mathbb{R}^{m,n}$ e sia

$$\mathbf{A} = \mathbf{U}\mathbf{S}\mathbf{V}^T$$

la sua decomposizione ai valori singolari, con

$$s_1 \geq s_2 \geq \cdots \geq s_r > s_{r+1} = \cdots = s_p = 0$$

e sia k un intero positivo minore o uguale a r. Indicando con

$$\mathbf{A}_k = \sum_{i=1}^{k} s_i \mathbf{u}_i \mathbf{v}_i^T$$

e

$$\mathcal{B}_k = \{\mathbf{B} \in \mathbb{R}^{m,n} : \text{rango di } \mathbf{B} \text{ è } k\}$$

si ha

$$\min_{\mathbf{B} \in \mathcal{B}_k} \|\mathbf{A} - \mathbf{B}\|_2 = \|\mathbf{A} - \mathbf{A}_k\|_2 = s_{k+1}$$

Il precedente teorema ci dice quindi che la matrice $\mathbf{A}_k$ rappresenta la migliore approssimazione di rango k in norma 2 della matrice $\mathbf{A}$.
Inoltre dal momento che la distanza di $\mathbf{A}$ dall'insieme delle matrici di rango k è definita $d(\mathbf{A}, \mathcal{B}_k) = \inf_{\mathbf{B} \in \mathcal{B}_k} \|\mathbf{A} - \mathbf{B}\|$ in norma 2 tale distanza è s_{k+1}.

4.2 Applicazioni della SVD

4.2.1 Calcolo del rango di una matrice

La decomposizione SVD è utile per determinare il rango di una matrice. Infatti in aritmetica esatta il numero di valori singolari maggiori di zero è proprio il rango della nostra matrice. Tuttavia in aritmetica non esatta può accadere che alcuni valori che dovrebbero essere zero siano in realtà numeri molto piccoli, prossimi alla zero, ma comunque diversi da quest'ultimo. Quindi numericamente i valori singolari devono essere selezioni rispetto ad una certa tolleranza `tol` e quindi si definisce **rango numerico** il numero dei valori singolari maggiori di una tolleranza fissata.
In MatLab la funzione `rank(A)` calcola proprio il rango numerico.

MATLAB
```
r = rank(A); % Restituisce il rango numerico rispetto ad una
tolleranza di default che dipende dalla matrice A e dalla
precisione di macchina
r = rank(A,tol); % Restituisce il rango numerico alla
tolleranza tol
```

4.2.2 Risoluzione di un sistema lineare

Dato un sistema lineare $\mathbf{Ax} = \mathbf{b}$ se si conosce la scomposizione SVD di $\mathbf{A}$ di rango n n e non singolare è possibile ottenere la soluzione $\mathbf{x}$ tramite la seguente procedura

$$\mathbf{USV}^{\mathrm{T}}\mathbf{x} = \mathbf{b} \Rightarrow \underbrace{\mathbf{SV}^{\mathrm{T}}\mathbf{x}}_{\mathbf{y}} = \mathbf{U}^{\mathrm{T}}\mathbf{b} \Rightarrow \begin{cases} \mathbf{Sy} = \mathbf{U}^{\mathrm{T}}\mathbf{b} \Rightarrow \mathbf{y} \\ \mathbf{V}^{\mathrm{T}}\mathbf{x} = \mathbf{y} \Rightarrow \mathbf{x} = \mathbf{Vy} \end{cases}$$

ovvero risolvendo un sistema diagonale e facendo due pordotti matrice-vettore. Tale procedura è vantaggiosa solo se si conosce già la scomposizione SVD della matrice per altri motivi dal momento che il costo computazionale di tale scomposizione è $O(32/3n^2)$ di gran lunga superiore a tutte le altre scomposizioni.
Tale procedura è però sempre ben condizionata e quindi a volte si preferisce alla classica $\mathbf{PA} = \mathbf{LU}$

4.2.3 Risoluzione di un sistema lineare sovra-determinato

Riprendiamo ora la trattazione sui sistemi sovra-determinati iniziata nel capitolo precedente. In particolar modo ci concentriamo sul caso 2) Matrice $\mathbf{A}$ di rango non massimo.
Consideriamo il sistema lineare $\mathbf{Ax} = \mathbf{b}$ con $\mathbf{A} \in \mathbb{R}^{m,n}$ con $m \geq n$ e con di rango $rk(\mathbf{A}) = r \leq n$.
Come abbiamo descritto in modo più approfondito nel capitolo precedente si dice soluzione del sistema nel senso dei minimi quadrati il vettore $\mathbf{x}^*$ che ha norma minima rispetto ai vettori $\mathbf{y}$ che soddisfano

$$\|\mathbf{Ay} - \mathbf{b}\|_2 = \min_{\mathbf{x} \in \mathbb{R}^n} \|\mathbf{Ax} - \mathbf{b}\|_2$$

Per calcolare $\mathbf{x}^*$ useremo un procedimento basato sulla scomposizione SVD, $\mathbf{A} = \mathbf{USV^T}$.
Come nel caso 1) $\mathbf{A}$ di rango massimo possiamo minimizzare il quadrato della norma invece che la norma essendo quest'ultima sempre positiva ed essendo la funzione x^2 strettamente crescente in $[0, +\infty)$.
Per il lemma **??** sappiamo che se $\mathbf{U}$ è ortogonale allora $\|\mathbf{Uy}\|_2 = \|\mathbf{y}\|_2$.
Quindi

$$\|\mathbf{Ax} - \mathbf{b}\|_2^2 = \left\|\mathbf{U}^\top(\mathbf{Ax} - \mathbf{b})\right\|_2^2 = \left\|\mathbf{U}^\top\mathbf{AVV}^\top\mathbf{x} - \mathbf{U}^\top\mathbf{b}\right\|_2^2 = \|\mathbf{Sy} - \mathbf{c}\|_2^2$$

$$\text{con } \mathbf{y} := \mathbf{V}^\top\mathbf{x} \quad \text{e} \quad \mathbf{c} := \mathbf{U}^\top\mathbf{b}.$$

Dal momento che r è il numero di valori singolari non nulli di $\mathbf{A}$ possiamo riscrivere $\mathbf{S}$, $\mathbf{y}$, $\mathbf{c}$ come

$$\mathbf{S} = \begin{pmatrix} \tilde{\mathbf{S}} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{pmatrix}, \quad \tilde{\mathbf{S}} \in \mathbb{R}^{r,r}$$

$$\mathbf{y} = \begin{pmatrix} \mathbf{y}_1 \\ \mathbf{y}_2 \end{pmatrix}, \quad \mathbf{c} = \begin{pmatrix} \mathbf{c}_1 \\ \mathbf{c}_2 \end{pmatrix}, \quad \mathbf{y}_1, \mathbf{c}_1 \in \mathbb{R}^r, \quad \mathbf{y}_2 \in \mathbb{R}^{n-r}, \quad \mathbf{c}_2 \in \mathbb{R}^{m-r}$$

e quindi il nostro problema diventa

$$\|\mathbf{Ax} - \mathbf{b}\|_2^2 = \|\mathbf{Sy} - \mathbf{c}\|_2^2 = \left\|\begin{pmatrix} \tilde{\mathbf{S}}\mathbf{y}_1 - \mathbf{c}_1 \\ -\mathbf{c}_2 \end{pmatrix}\right\|_2^2 = \left\|\tilde{\mathbf{S}}\mathbf{y}_1 - \mathbf{c}_1\right\|_2^2 + \|\mathbf{c}_2\|_2^2$$

e quindi $\|\mathbf{Ax} - \mathbf{b}\|_2^2$ è minima se $\left\|\tilde{\mathbf{S}}\mathbf{y}_1 - \mathbf{c}_1\right\|_2^2 = 0$ ovvero se $\tilde{\mathbf{S}}\mathbf{y}_1 - \mathbf{c}_1 = 0$ ammette soluzione.
Dal momento che $\tilde{\mathbf{S}}$ è non singolare e diagonale il sistema ammette una e una sola soluzione.
Il lettore più istruito infatti saprà che data un'applicazione lineare (endomorfismo) $f : \mathbb{R}^r \to \mathbb{R}^r \quad \mathbf{y_1} \to \tilde{\mathbf{S}}\mathbf{y_1}$ risolvere il sistema $\tilde{\mathbf{S}}\mathbf{y}_1 = \mathbf{c}_1$ equivale a trovare la controimmagine $f^{-1}(\mathbf{c_1})$ del vettore $\mathbf{c_1}$.
Se $\tilde{\mathbf{S}}$ non è singolare ovvero se $\mathrm{rk}(\tilde{\mathbf{S}}) = \dim(\mathrm{Im}(f)) = r$ per il teorema delle dimensioni $\dim(\ker(f)) = 0$ e quindi l'applicazione è iniettiva. Inoltre $\dim(\mathrm{Im}(f)) = \dim(\mathbb{R}^r)$ e quindi $\mathrm{Im}(f) = \mathbb{R}^r$ e quindi l'applicazione è anche suriettiva. Esiste quindi unica $f^{-1}(\mathbf{c_1})$ che chiameremo $\mathbf{y_1^*}$.

Denotando con $(\mathbf{y_1})_i$ le componenti di $\mathbf{y_1}$ e con $\mathbf{u_i}$ la i-esima colonna di $\mathbf{U}$ imponendo

$$\left\|\tilde{\mathbf{S}}\mathbf{y}_1 - \mathbf{c}_1\right\|_2^2 = \sum_{i=1}^{r} \left(s_i(\mathbf{y}_1)_i - \mathbf{u}_i^\top \mathbf{b}\right)^2 = 0,$$

si ricava

$$(\mathbf{y}_1^*)_i = \frac{\mathbf{u}_i^\top \mathbf{b}}{s_i}, \quad i = 1, \ldots, r$$

Ogni $\mathbf{y}$ le cui prime r componenti coincidono con quelle di $\mathbf{y_1}^*$ minimizza $\|\mathbf{Ax} - \mathbf{b}\|_2$. Siamo interessati però alla soluzione di norma minima cioè ad che chiameremo $\mathbf{y}^*$, avente tutte le componenti nulle dalla $r+1$ in poi ovvero

$$y_i^* = \begin{cases} \dfrac{\mathbf{u}_i^\top \mathbf{b}}{s_i}, & i = 1, \ldots, r \\ 0, & i = r+1, \ldots, n \end{cases}$$

Dal momento che $\mathbf{x} = \mathbf{Vy}$ la soluzione nel senso dei minimi quadrati del sistema $\mathbf{Ax} = \mathbf{b}$ è

$$\mathbf{x}^* = \mathbf{Vy}^* = \sum_{i=1}^{r} y_i^* \, \mathbf{v}_i$$

e dal momento che $\mathbf{V}$ è ortogonale $\|\mathbf{x}^*\|_2 = \|\mathbf{Vy}^*\|_2 = \|\mathbf{y}^*\|_2$, quindi $\mathbf{x}^*$ ha norma minima. Definiamo

$$\mathbf{S}^+ \in \mathbb{R}^{n,m} : \quad (\mathbf{S}^+)_{ij} = \begin{cases} \dfrac{1}{s_i}, & i = j = 1, \ldots, r \\ 0, & i = j = r+1, \ldots, n, \ i \neq j \end{cases}$$

allora

$$\mathbf{y}^* = \mathbf{S}^+\mathbf{U}^\top \mathbf{b}$$
$$\mathbf{x}^* = \mathbf{VS}^+\mathbf{U}^\top \mathbf{b}$$

Definizione 4.2.1. Sia $\mathbf{A}^+ = \mathbf{VS}^+\mathbf{U}^\top$. La matrice $\mathbf{A}^+$ viene definita **pseudo-inversa di Moore-Penrose** o **inversa generalizzata** di $\mathbf{A}$

La soluzione nel senso dei minimi quadrati del sistema $\mathbf{Ax} = \mathbf{b}$ è data quindi da

$$\mathbf{x}^* = \mathbf{A}^+\mathbf{b}$$

Si osservi che se $\mathbf{A} \in \mathbb{R}^{n,n}$ e $\operatorname{rank}(\mathbf{A}) = n$, $\quad \mathbf{A}^+ = \mathbf{A}^{-1}$.
Infatti se $\mathbf{A} = \mathbf{USV^T}$ e $\mathbf{A}$ è di rango pieno e quadrata allora

- $\mathbf{U}$ e $\mathbf{V}$ sono ortogonali $n \times n$
- $\mathbf{S} \in \mathbb{R}^{n,n}$ è diagonale e invertibile in quanto $det(S) = s_1 \cdot \ldots \cdot s_n \neq 0$ essendo $s_1, \ldots, s_n$ i suoi valori singolari

La pseudo-inversa è $\mathbf{A}^+ = \mathbf{V}\mathbf{S}^+\mathbf{U}^\mathbf{T}$ ma $\mathbf{S}^+ = \mathbf{S}^{-\mathbf{1}}$ essendo l'inversa di una matrice diagonale un'altra matrice diagonale del tipo $\text{diag}(1/a_1, \ldots, 1/a_n)$ e non avendo $\mathbf{S}^+$ entrate nulle sulla diagonale.
Risulta quindi che

$$\mathbf{A}^+ = \mathbf{V}\mathbf{S}^{-1}\mathbf{U}^\top = \left(\mathbf{U}\mathbf{S}\mathbf{V}^\top\right)^{-1} = \mathbf{A}^{-1}$$

In MatLab sono presenti i seguenti comandi che implementano quanto detto fino a questo punto.

MATLAB

```
X = pinv(A) % Restituisce la pseudo-inversa di A a partire
dalla sua SVD. I valori singolari sotto una certa tolleranza
vengono trattati come zeri.
X = pinv(A,tol) % Permette di decidere la tolleranza
x = pinv(A)*b % Nel caso di un sistema lineare
sovradeterminato Ax= b, con A matrice rettangolare di
dimensioni m ×n con m >n, fornisce (utilizzando la procedura
precedentemente descritta) la soluzione x nel senso dei
minimi quadrati del sistema Ax= b avente norma euclidea
minima.
```

Si rammenta al lettore che non è possibile risolvere un sistema-sovradeterminato con matrice $\mathbf{A}$ di rango non massimo con \ nel senso appena descritto. Infatti restituirà la soluzione con maggior numero di entrate nulle e non quella di minima norma.

4.2.4 Risoluzione di un sistema mal condizionato

Nel caso di un sistema mal condizionato al posto della fattorizzazione **PA=LU** possiamo usare approssimare la matrice $\mathbf{A}$ con una matrice di rango inferiore trascurando i valori singolari sotto una determinata tolleranza e risolvere il sistema nel senso dei minimi quadrati.

4.2.5 Risoluzione di un sistema lineare sotto-determinato

Si consideri il sistema lineare $\mathbf{Ax} = \mathbf{b}$ con $\mathbf{A}$ di dimensione $m \times n$, con $m < n$. Si supponga che il rango di $\mathbf{A}$ sia uguale ad $r = m$ e che $\mathbf{A}$ ammetta la seguente decomposizione ai valori singolari $\mathbf{A} = \mathbf{USV}^\top$.
Il sistema ammette infinite soluzioni che dipendono da $n - r$ parametri liberi (Rouché-Capelli).

Si ha

$$\mathbf{Ax} = \mathbf{b} \Longrightarrow \mathbf{S}\underbrace{\mathbf{V}^\top\mathbf{x}}_{\mathbf{y}} = \mathbf{U}^\top\mathbf{b} \Longrightarrow \begin{cases} \mathbf{Sy} = \mathbf{U}^\top\mathbf{b} \\ \mathbf{V}^\top\mathbf{x} = \mathbf{y} \end{cases}$$

Riscrivendo $\mathbf{S}$ nella forma

$$\mathbf{S} = \begin{pmatrix} \tilde{\mathbf{S}} & \mathbf{0} \end{pmatrix}, \quad \tilde{\mathbf{S}} \in \mathbb{R}^{m,m}$$

il sistema $\mathbf{Sy} = \mathbf{d}$, ove $\mathbf{d} := \mathbf{U}^\top\mathbf{b}$, ammette come soluzione di norma euclidea minima il vettore

$$y_i^* = \frac{d_i}{s_i}, \quad i = 1, \ldots, m, \qquad y_i^* = 0, \quad i = m+1, \ldots, n.$$

Inoltre, per l'ortogonalità di $\mathbf{V}$, la norma euclidea di $\mathbf{x}$ è minima se e solo se la norma euclidea di $\mathbf{y} = \mathbf{V}^\top\mathbf{x}$ è minima, pertanto la soluzione $\mathbf{x}^*$ del sistema assegnato è data da

$$\mathbf{x}^* = \mathbf{Vy}^*.$$

Capitolo 5

Autovalori

In questo capitolo daremo in primo luogo un breve ripasso sulla teoria degli autovalori (non vuole essere una trattazione esaustiva) di una matrice per poi passare allo studio del condizionamento del problema del calcolo degli stessi e descriveremo alcuni basici metodi numerici per la loro determinazione.

5.1 Cenni alla teoria degli autovalori e autovettori

Definizione 5.1.1 (Autovalori e autovettori)**.** Data una matrice $\mathbf{A} \in \mathbb{R}^{n,n}$ si dice **autovalore** di $\mathbf{A}$ un numero λ tale che esiste un vettore $\mathbf{v} \in \mathbb{R}^n$ che soddisfi:

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$$

In tal caso $\mathbf{v}$ è detto **autovettore** di $\mathbf{A}$ relativo a λ.

Nota che λ è autovalore di $\mathbf{A}$ solo se la matrice $\mathbf{A} - \lambda\mathbf{I}$ è singolare. Infatti se $\mathbf{A} - \lambda\mathbf{I}$ non fosse singolare allora esisterebbe un solo vettore (il vettore nullo) tale che $(\mathbf{A} - \lambda\mathbf{I})\mathbf{v} = 0$.
Quindi gli autovalori di $\mathbf{A}$ coincidono con le soluzioni dell'equazione caratteristica

$$p(\lambda) = det(\mathbf{A} - \lambda\mathbf{I}) = 0$$

$p(\lambda)$ è detto polinomio caratteristico di $\mathbf{A}$.
Quindi una matrice $\mathbf{A} \in \mathbb{R}^{n,n}$ ha esattamente n autovalori in $\mathbb{C}$ ed essendo $\mathbf{A}$ a coefficienti reali allora se ammette una soluzione complessa ammette anche

la sua coniugata.

Le matrici e i loro autovalori godono, tra le altre, di queste due proprietà:

1. Se $\lambda_1, \cdots, \lambda_n$ sono gli autovalori di $\mathbf{A}$ allora $\lambda_1^k, \cdots, \lambda_n^k$ sono gli autovalori di $\mathbf{A^k}$.

2. Se $\lambda_1, \cdots, \lambda_n$ sono gli autovalori di $\mathbf{A}$ allora $\lambda_1^{-1}, \cdots, \lambda_n^{-1}$ sono gli autovalori di $\mathbf{A^{-1}}$, inversa di $\mathbf{A}$.

La definizione di autovalore-autovettore

$$\mathbf{A}\mathbf{x}_i = \lambda_i \mathbf{x}_i, \quad i = 1, \ldots, n$$

consente di scrivere l'identità

$$(\mathbf{A}\mathbf{x}_1, \mathbf{A}\mathbf{x}_2, \ldots, \mathbf{A}\mathbf{x}_n) = (\lambda_1\mathbf{x}_1, \lambda_2\mathbf{x}_2, \ldots, \lambda_n\mathbf{x}_n)$$

ovvero

$$\mathbf{AX} = \mathbf{XD}$$

con

$$\mathbf{X} = (\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n), \quad \mathbf{D} = \begin{pmatrix} \lambda_1 & & 0 \\ & \lambda_2 & \\ 0 & & \lambda_n \end{pmatrix}$$

In MatLab esistono comandi per estrarre gli autovalori e tali matrici:

```
MATLAB d = eig(A) % Restituisce il vettore d che
contiene gli autovalori di A
[X, D] = eig(A) % Restituisce X e D definitivi
precedentemente
```

Definizione 5.1.2. Data una matrice $\mathbf{A}$ di ordine n e un vettore non nullo $\mathbf{x}$ di dimensione n si definisce **quoziente di Rayleigh** il numero

$$r_A(\mathbf{x}) = \frac{\mathbf{x^H A x}}{\mathbf{x^H x}}$$

dove $\mathbf{x^H}$ è il vettore trasposto coniugato ovvero il vettore i cui elementi $\bar{x}_j$ sono i coniugati degli elementi x_j di $\mathbf{x}$. Se gli elementi di $\mathbf{x}$ sono tutti reali allora $\mathbf{x^H} = \mathbf{x^T}$

Si osservi che se $\mathbf{x}$ è un autovettore relativo a λ allora $r_A(x) = \lambda$.

Per una matrice $\mathbf{A}$ simmetrica definita positiva ad entrate reali il prodotto $\mathbf{x^T A x}$ è positivo ed essendo $\mathbf{x^T x} = \|\mathbf{x}\|_\mathbf{2}^\mathbf{2}\ r_A(x)$ risulta positivo. Ne segue che una matrice simmetrica definita positiva ha solo autovalori positivi.

Abbiamo così individuato due metodi "teorici" per individuare gli autovalori di una matrice e un autovettore da un autovalore.

Definizione 5.1.3 (Raggio spettrale)**.** Si definisce **raggio spettrale** della matrice $\mathbf{A}$ di ordine n il modulo del massimo autovalore di $\mathbf{A}$ cioè:

$$\rho(\mathbf{A}) = \max_{i=1,\dots,n} \{|\lambda_i| \ : \ \lambda_i \text{ è autovalore di } \mathbf{A}\}$$

```
MATLAB rA = max(abs(eig(A))) % Calcola il raggio
spettrale di A
```

Definizione 5.1.4 (Norma spettrale)**.** Si definisce **norma 2** o **norma spettrale** di $\mathbf{A} \in \mathbb{R}^{m,n}$ il numero reale

$$\|\mathbf{A}\|_2 = \sqrt{\rho(\mathbf{A^T A})}$$

Se $\mathbf{A} \in \mathbb{R}^{n,n}$ è simmetrica allora $\|\mathbf{A}\|_2 = \rho(\mathbf{A})$

```
MATLAB norm(A,2) = norm(A) % restituiscono la norma
spettrale di A
```

Definizione 5.1.5. Due matrici $\mathbf{A}$ e $\mathbf{B}$ si dicono simili se esiste una matrice $\mathbf{S}$ tale che

$$\mathbf{S^{-1} A S} = \mathbf{B}$$

Teorema 5.1.1. Due matrici simili $\mathbf{A}$ e $\mathbf{B}$ hanno stessi autovalori.

Definizione 5.1.6. Una matrice $\mathbf{A}$ con autovalori $\lambda_i, i = 1, \cdots n$ si dice **diagonalizzabile** se è simile alla matrice diagonale $\mathbf{D} = diag(\lambda_1, \cdots, \lambda_n)$ ovvero se esiste una matrice di ordine n e non singolare $\mathbf{S}$ tale che

$$\mathbf{S^{-1} A S} = \mathbf{D} = diag(\lambda_1, \cdots, \lambda_n)$$

Si noti che se $\mathbf{A}$ è diagonalizzabile e $\mathbf{S^{-1} A S} = \mathbf{D}$ allora

$$AS = SD$$

e quindi le colonne della matrice $\mathbf{S}$ sono autovettori di $\mathbf{A}$

5.2 Condizionamento del calcolo degli autovalori

Vediamo ora il condizionamento del problema del calcolo degli autovalori, ovvero si analizza come gli errori di arrotondamento o di misurazione su $\mathbf{A}$ si riflettono sui valori degli autovalori. Nello specifico vedremo il condizionamento del calcolo di autovalori di matrici diagonalizzabili.

Teorema 5.2.1 (Bauer-Fike)**.** Sia $\mathbf{A}$ una matrice **diagonalizzabile** e sia $\mathbf{S}$ invertibile tale che

$$\mathbf{S}^{-1}\mathbf{A}\mathbf{S} = \mathbf{D} \quad \text{con } \mathbf{D} = \text{diag}(\lambda_1, \lambda_2, \ldots, \lambda_n).$$

Sia $\widetilde{\mathbf{A}}$ una perturbazione della matrice $\mathbf{A}$ e sia $\widetilde{\lambda}$ un autovalore di $\widetilde{\mathbf{A}}$. Allora

$$\min_{1\leq i\leq n} |\widetilde{\lambda} - \lambda_i| \leq K(\mathbf{S})\|\mathbf{A} - \widetilde{\mathbf{A}}\|$$

dove $K(\mathbf{S}) = \|\mathbf{S}\|\,\|\mathbf{S}^{-1}\|$ e $\|\cdot\|$ indica una qualsiasi delle norme di matrice $\|\cdot\|_p$, con $p = 1, 2, \infty$. $K(\mathbf{S})$ si definisce numero di condizionamento del calcolo degli autovalori. Se $K(\mathbf{S}) = 1$ allora il calcolo è ben condizionato.

Valgono le seguenti affermazioni

1. Se $\mathbf{A}$ è simmetrica il problema di calcolo degli autovalori è ben condizionato infatti

$$K_2(\mathbf{S}) = \|\mathbf{S}\|_2\|\mathbf{S^T}\|_2 = \sqrt{\rho(\mathbf{S^T S})}\sqrt{\rho(\mathbf{S S^T})} = 1$$

2. Il condizionamento del calcolo degli autovalori di una matrice diagonalizzabile $\mathbf{A}$ non dipende dal numero di condizionamento di $\mathbf{A}$ (K(A)) ma dal numero di condizionamento di $\mathbf{S}$ (K(S))

3. Il calcolo degli autovalori di una matrice mal condizionata non è necessariamente un problema mal condizionato. Ad esempio la matrice di Hilbert, fortemente mal condizionata, essendo simmetrica ha condizionamento 1 per il calcolo dei suoi autovalori.

MATLAB
```
[S,D ] = eig(A);
condizionamento = cond(S)
```

5.3 Metodi numerici per il calcolo degli autovalori

Per il calcolo degli autovalori di una matrice $\mathbf{A} \in \mathbb{R}^{n,n}$ trovare le radici del polinomio caratteristico non è una procedura conveniente ne in termini di costo ne in termini di accuratezza. Esistono altri metodici numerici per il calcolo che si classificano in due gruppi:

1. Metodi per l'approssimazione di un solo autovalore
2. Metodi per l'approssimazione simultanea di tutti gli autovalori

5.3.1 Metodi per l'approssimazione di un solo autovalore

Questi metodi appartengono alla classe dei problemi iterativi. Generano una successione di valori che, sotto determinate condizioni, convergono all'autovalore cercato.

Metodo delle potenze

Il metodo delle potenze consente di calcolare l'autovalore di modulo massimo di una matrice $\mathbf{A}$ e quindi il suo raggio spettrale $\rho(\mathbf{A})$. Per questo metodo supponiamo che $\mathbf{A}$ sia diagonalizzabile e che esista un solo autovalore di modulo massimo.

Cominciamo considerando una base di $(x_1, \cdots, x_n)$ di autovettori.
Tale base esiste dal momento che abbiamo supposto $\mathbf{A}$ diagonalizzabile (gli autospazi sono abbastanza "grandi").
Scelgo $\mathbf{z} \neq 0_n \in \mathbb{R}^n$ per cui

$$\mathbf{z} = \alpha_1 \mathbf{x_1} + \cdots + \alpha_n \mathbf{x_n} \Rightarrow \mathbf{Az} = \alpha_1 \mathbf{Ax_1} + \cdots + \alpha_n \mathbf{Ax_n}$$

e dal momento che $\mathbf{Ax_i} = \lambda_\mathbf{i} \mathbf{x_i} \quad i = 1, \cdots, n$ allora

$$\mathbf{Az} = \alpha_1 \lambda_1 \mathbf{x_1} + \cdots + \alpha_n \lambda_n \mathbf{x_n}$$

Di conseguenza

$$\mathbf{A^2 z} = \alpha_1 \lambda_1 \mathbf{Ax_1} + \cdots + \alpha_n \lambda_n \mathbf{Ax_n} = \alpha_1 \lambda_1^2 \mathbf{x_1} + \cdots + \alpha_n \lambda_n^2 \mathbf{x_n}$$

e iterando m volte si ottiene

$$\mathbf{A^m z} = \alpha_1 \lambda_1^m \mathbf{x_1} + \cdots + \alpha_n \lambda_n^m \mathbf{x_n} = \lambda_1^m \left(\alpha_1 \mathbf{x_1} + \cdots + \alpha_n \left(\frac{\lambda_n}{\lambda_1} \right)^m \mathbf{x_n} \right)$$

Si ha che

$$\lim_{m \to +\infty} \lambda_1^m \left(\alpha_1 \mathbf{x}_1 + \cdots + \alpha_n \left(\frac{\lambda_n}{\lambda_1} \right)^m \mathbf{x}_n \right) = \lambda_1^m \alpha_1 \mathbf{x_1}$$

Dal momento che un autovettore è parte di uno spazio vettoriale (autospazio) formato da tutti autovettori relativi allo stesso autovalore e che la dimensione di tali spazi (per una matrice diagonalizzabile) coincide con la molteplicità di λ come radice del polinomio caratteristico allora $\lambda_1^m \alpha_1 \mathbf{x_1}$ è ancora un autovettore relativo a λ_1.

Definiamo quindi

$$\mathbf{y^{(m)}} = \mathbf{A^m z} \sim \lambda_\mathbf{1}^\mathbf{m} \alpha_\mathbf{1} \mathbf{x_1}, \quad m \to +\infty$$

Il quoziente di Rayleigh di $\mathbf{y^{(m)}}$ allora è proprio λ_1:

$$\lambda_1 = \frac{\mathbf{y^{(m)^T} A y^{(m)}}}{\mathbf{y^{(m)^T} y^{(m)}}}$$

La procedura descritta nonostante sia impeccabile da un punto di vista teorico ha due fondamentali problemi da un punto di vista numerico. Infatti $\mathbf{y^{(m)}}$ potrebbe non essere calcolabile perché

1. λ_1^m per $|\lambda 1| >> 1$ potrebbe dare `overflow`
2. λ_1^m per $|\lambda 1| << 1$ potrebbe dare `underflow`

La risoluzione a questo problema consiste nel normalizzare $\mathbf{y^{(k)}}$ ad ogni iterazione k, ovvero si definisce

$$\omega^{(\mathbf{k})} = \mathbf{y^{(k)}} / \|\mathbf{y^{(k)}}\|_\mathbf{2}$$

e

$$\mathbf{y^{(k+1)}} = \mathbf{A} \omega^{(\mathbf{k})}$$

Di seguito riportiamo un'implementazione di questo algoritmo in MatLab

MATLAB

```
function [lambdaMax, w, m] = potenze(A, y, tol, mMax)

w = y/norm(y);% Normalizza il vettore in input per evitare
overflow/underflow alla prima iterazione

lambda = 0;

for m = 1:mMax

y = A*w; % Allinea w ad un autovettore di lamdaMax

lambdaMax = w'*y ;% Dal momento che w'*w = 1 e l'attuale
y=A*w utilizziamo w dell'iterazione precedente per
risparmiare calcoli costosi come i prodotti tra matrici

w = y/norm(y); % Rinormalizziamo y per usarlo
nell'iterazione successiva

% Controlla se l'errore relativo scende sotto la tolleranza
specificata
if abs(lambdaMax-lambda) <= tol*abs(lambdaMax)
break
end

% Assegna il nuovo miglior valore di lambda lambda =
lambdaMax;

end
```

Segnaliamo che

1. Anche se il vettore iniziale $\mathbf{y^{(0)}}$ ha componente nulla rispetto a $\mathbf{x_1}$ grazie (questa volta) agli errori di arrotondamento dopo qualche iterazione compare una componente.

2. Se $\lambda_1 = \cdots = \lambda_k$ e $|\lambda_1| \geq \cdots \geq |\lambda_{k+1}|$ il metodo converge molto lentamente ma continua a convergere a λ_1. In questo caso si usa il metodo per una stima iniziale di λ_1 per poi usare metodi più veloci

3. Se $\lambda_1 = -\lambda_2$ allora il metodo non converge

4. Se $\lambda_1 e \lambda_2$ sono complessi coniugati, cioè sono due autovalori di modulo massimo, il metodo non converge perchè non può essere applicato ai complessi ma solo ai reali

5. Per una generica matrice la velocità di convergenza del metodo è la stessa con la quale il rapporto $|\lambda_1|^m/|\lambda_2|^m$ converge a 0.

Metodo delle potenze inverse

Il metodo delle potenze inverse è una variante del metodo delle potenze che permette di approssimare un qualunque autovalore λ di $\mathbf{A}$ purchè se ne conosca già un'approssimazione più rozza p.
Infatti dal fatto che $\mathbf{Ax} = \lambda\mathbf{x}$ possiamo dedurre

$$\mathbf{Ax} - p\mathbf{x} = \lambda\mathbf{x} - p\mathbf{x} \Rightarrow (\mathbf{A} - p\mathbf{I})\mathbf{x} = (\lambda - p)\mathbf{x}$$

ovvero che $\mu = (\lambda - p)$ è autovalore della matrice $\mathbf{M} = (\mathbf{A} - p\mathbf{I})$.
Per $\mathbf{M}$ invertibile si ha

$$\mathbf{M^{-1}x} = \frac{1}{\mu}\mathbf{x}$$

e dal momento che

$$\frac{1}{\mu} = \frac{1}{\lambda - p}$$

se λ è molto vicino a p, $1/\mu$ risulta essere l'autovalore di modulo massimo di $\mathbf{M^{-1}}$. Possiamo quindi calcolarlo con il metodo delle potenze visto in precedenza con un piccolo accorgimento.
Dal momento che ad ogni passo dovremmo calcolare

$$\mathbf{y^{(k+1)}} = (\mathbf{A} - \mathbf{pI})^{\mathbf{-1}}\omega^{\mathbf{(k)}}$$

che equivale a risolvere il sistema

$$(\mathbf{A} - \mathbf{pI})\mathbf{y^{(k+1)}} = \omega^{\mathbf{(k)}}$$

possiamo calcolare la fattorizzazione PA = LU di $(\mathbf{A} - \mathbf{pI})$ ed usarla ad ogni iterazione per risolvere il sistema triangolare superiore.

MATLAB

```
function [lambdaP,w,m] = potenzeInverse(A,p,y,tol,mMax)

n = size(A);

w = y/norm(y);

lambda = p;% Definisce la prima approssimazione di lambda

[L,U,P] = lu(A-p*eye(n));

for m = 1:mMax

z = L\(P*w);

y = U\z;

lambdaP = p+1/(w'*y);% Calcola la nuova approssimazione
di lambda

w = y/norm(y);

if abs(lambdaP-lambda) <= tol*abs(lambdaP)
break
end

lambda = lambdaP;

end
```

Seguono considerazioni sul metodo

1. se $p = 0$ il metodo consente di approssimare l'autovalore di modulo minimo nell'ipotesi che sia unico

2. Il valore di p potrebbe essere modificato durante le iterazioni per far convergere prima il metodo,ma velocità è solo apparente. Infatti non sarebbe più possibile utilizzare la PA=LU e quindi il costo computazionale salirebbe notevolmente.

In MatLab i seguenti comandi che ci permettono di stimare uno o più autovalori si basano su questo metodo

MATLAB
```
[X,D] = eigs(A,k) % Restituisce in D e in X rispettivamente
i k autovalori di A di modulo più grande e i loro
autovettori

[X,D] = eigs(A,k) % Restituisce in D e in X rispettivamente
i k autovalori di A più vicini a p
```

Metodo QR

Il metodo più efficiente per l'approssimazione di tutti gli autovalori di una matrice si basa sulla fattorizzazione $\mathbf{A} = \mathbf{QR}$.
Descriveremo solo una forma elementare del metodo che però non garantisce la convergenza agli autovalori di $\mathbf{A}$ in ogni circostanza.
Il metodo consiste nel generare una successione di matrici simili tra loro.
Sia $\mathbf{A_1} = \mathbf{A}$. Ad ogni passo k calcoliamo la fattorizzazione QR della matrice $\mathbf{A_k} = \mathbf{Q_k R_k}$ e definiamo la matrice $\mathbf{A_{k+1}}$ come:

$$\mathbf{A_{k+1}} = \mathbf{R_k Q_k}$$

Dal momento che

$$\mathbf{A_{k+1}} = \mathbf{R_k Q_k} = \mathbf{Q_k^T Q_k R_k Q_k} = \mathbf{Q_k^T A_k Q_k}$$

le matrici $\mathbf{A_k}$ e $\mathbf{A_{k+1}}$ risultano simili e quindi hanno gli stessi autovalori. A catena quindi $\mathbf{A_{k+1}}$ risulta simile a tutte le $\mathbf{A_i}$ per $i = 1 \cdots k$ e quindi anche ad $\mathbf{A}$. Inoltre se $\mathbf{A}$ è simmetrica lo sarà anche $\mathbf{A_{k+1}}$.
La successione di matrici per $k \to \infty$ converge ad una matrice $\mathbf{A}_\infty$ con le seguenti proprietà:

1. Se $\mathbf{A}$ è simmetrica allora $\mathbf{A}_\infty$ è diagonale con gli autovalori di $\mathbf{A}$ sulla diagonale

2. Se $\mathbf{A}$ non è simmetrica ma ha tutti autovalori reali allora $\mathbf{A}_\infty$ è triangolare superiore con gli autovalori di $\mathbf{A}$ sulla diagonale

3. Se $\mathbf{A}$ non è simmetrica e ha alcuni autovalori complessi coniugati allora $\mathbf{A}_\infty$ è quasi triangolare superiore. Presenta sulla diagonale sia blocchi 1x1 che contengono i suoi autovalori reali sia blocchi 2x2. Gli autovalori di questi ultimi blocchi sono gli autovalori complessi coniugati della matrice $\mathbf{A}$

$$\begin{bmatrix} \lambda_1 & * & * & * & * & * & * \\ 0 & a_1 & a_2 & * & * & * & * \\ 0 & a_3 & a_4 & * & * & * & * \\ 0 & 0 & 0 & \lambda_2 & * & * & * \\ 0 & 0 & 0 & 0 & a_5 & a_6 & * \\ 0 & 0 & 0 & 0 & a_7 & a_8 & * \\ 0 & 0 & 0 & 0 & 0 & 0 & \lambda_3 \end{bmatrix}$$

Seguono osservazioni sul metodo.

1. La convergenza del metodo nella versione base è garantita solo se la matrice $\mathbf{A}$ ha autovalori tutti distinti in modulo (quindi non converge per autovalori complessi)

2. Il metodo QR sta alla base della funzione `eig` di MatLab

MATLAB

```
% Applicabile sono per matrici con autovalori distinti in
modulo function [d,m] = qrBase(A,tol,mMax)

for m = 1:mMax

[Q,R] = qr(A);

A = R*Q;

if norm(tril(A,-1),inf) <= tol
break
end

end

d = diag(A);
```

Capitolo 6

Analogie e differenze tra autovalori e valori singolari

Questo capitolo è un complemento al capitolo 4. Daremo alcune nozioni riguardo la decomposizione SVD per le quali era necessaria la conoscenza degli autovalori.

Teorema 6.0.1. Sia $\mathbf{A} \in \mathbb{R}^{m,n}$ e sia

$$\mathbf{A} = \mathbf{USV^T}$$

la sua decomposizione ai valori singolari. Se risulta che

$$s_1 \geq s_2 \geq \cdots \geq s_r \geq s_{r+1} = \cdots = s_p = 0$$

allora

1. $\lambda_i = s_i^2$, $i = 1 \cdots r$ sono gli autovalori della matrice $\mathbf{A^T A} \in \mathbb{R}^{n,n}$ (i restanti n-r autovalori sono nulli) e i corrispondenti autovettori formano la matrice ortogonale $\mathbf{V}$

2. $\lambda_i = s_i^2$, $i = 1 \cdots r$ sono gli autovalori della matrice $\mathbf{AA^T} \in \mathbb{R}^{m,m}$ (i restanti m-r autovalori sono nulli) e i corrispondenti autovettori formano la matrice ortogonale $\mathbf{U}$

3. Se $m = n$ e $\mathbf{A}$ è simmetrica allora $s_i = |\lambda_i|$, $i = 1 \cdots n$ e i vettori singolari destri e sinistri coincidono con gli autovettori di $\mathbf{A}$

Nonostante sia teoricamente possibile calcolare i valori singolari di $\mathbf{A}$ calcolando gli autovalori di $\mathbf{AA^T}$ questa procedura comporta una perdita di accuratezza

Calcolo del condizionamento di una matrice

Sia una matrice $\mathbf{A}$ di ordine n e di rango n e siano

$$s_1 \geq s_2 \geq \cdots \geq s_n > 0$$

i suoi valori singolari.
Per la proprietà 1. del precedente teorema segue che

$$\|\mathbf{A}\|_2 = \sqrt{\rho(\mathbf{A^T A}} = \sqrt{s_1^2} = s_1$$

Inoltre

$$\|\mathbf{A}^{-1}\|_2 = \sqrt{\rho((\mathbf{A^{-1}})^T \mathbf{A^{-1}})} = \sqrt{\rho((\mathbf{AA^T})^{-1})} = \frac{1}{s_n}$$

e quindi

$$K_2(\mathbf{A}) = \|\mathbf{A}\|_2 \|\mathbf{A^{-1}}\|_2 = \frac{s_1}{s_n}$$